



# Cloud Policy Cost and Security Simulator

Maryam Shahzad (2212396)  
Muhammad Abdur Rehman Khan (2212399)  
Wasfa Nauman Bhatti (2312462)

Supervised by: Mr. Hasnain Tahir

In partial fulfillment of requirement for the degree  
Bachelor of Science (Computer Science)

Department Computer Science  
Faculty of Computing and Engineering Sciences  
SZABIST University  
Islamabad, Pakistan

Spring 2026

# Project Overview

Cloud computing has radically changed the IT infrastructure of organizations by allowing Limitless scalability, flexibility and operational efficiency in industries such as healthcare to financial services. Nevertheless, this technological transformation presents a great challenge in the aspect of governance, cost management, and security operations that cannot be well tackled using the conventional IT education. The students and professionals joining the job market often have a theoretical certification and lack the practical experience in operating intricate cloud ecosystems which lead to expensive misconfigurations, vulnerabilities to security threats and ineffective use of resources in the production systems. This is a specific competency gap especially in developing countries like Pakistan, where schools have a tight budget to access a live cloud environment to make experiential learning possible.

Cloud Policy, Cost and Security Simulator turns out to be a holistic web based system designed to fill this essential gap by offering a risk free, AI driven Digital Twin experience in which users can enhance their practical skills in cloud management without risking their finances or data security. The proposed system is proposed as an improvement of existing educational tools as it will incorporate simulating synthetic resources to yield realistic utilization metrics based on real world data sets such as Google ClusterData2019 and Alibaba Cluster Trace v2018, natural language policy processing to facilitate governance-as-code, predictive cost forecasting with time series machine learning models trained on synthetic historical data. This simulator builds persistent, high-fidelity cloud environments in which learners can experiment with provisioning infrastructure, observe real time cost effects, simulated cyberattacks, and autonomous reactions by AI.

The platform includes a number of modules integrated together to provide a complete training experience in cloud operations. The Authentication and User Management module is used to offer secure JWT-based authentication including email verification workflows. Resource Simulation Engine provides virtual machines and databases with synthetic utilization metrics of CPU, Memory and I/O in real time. The AI driven Security Suite uses Deep Neural Networks, trained on the CICIDS2017 dataset to detect threats posed to simulated network traffic, including the ability to simulate DDoS and brute force attacks and an Autonomous Remediation Agent. The Predictive FinOps module applies the Time-Series Forecasting algorithms, such as ARIMA and LSTM, to produce 30-day cost predictions with confidence intervals.

The effective execution of this simulator brings about immense advantages to the academic and industrial sectors in Pakistan. To the students, the platform will seal the colossal disconnect between theory and practicality in the industry, which may provide

them with a secure environment to learn AIOps, Security, and Resource Utilization Optimization without the threat of inflicting any financial or security harm. This greatly enhances employability and technical confidence as it offers a taste of what tools that act like enterprise grade systems feel like. In the case of small and medium enterprises, it offers a pilot project to staff and predict costs without the cost of the cloud, enabling businesses to learn the ins and outs of cloud billing and governance before playing with real cloud vendors.

# Revision History

| Compiled By                | Checked By        | Date        | Description      | Version |
|----------------------------|-------------------|-------------|------------------|---------|
| Maryam Shahzad             | Mr. Hasnain Tahir | 08 Mar 2026 | Section 1.0      | 1.0     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 09 Mar 2026 | Section 1.1      | 1.1     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 11 Feb 2026 | Section 1.2, 1.3 | 1.2     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 13 Feb 2026 | Section 1.4      | 1.3     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 15 Mar 2026 | Section 1.5      | 1.4     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 17 Mar 2026 | Section 2.0      | 1.5     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 19 Mar 2026 | Section 2.1, 2.2 | 1.6     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 21 Mar 2026 | Section 2.3      | 1.7     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 23 Mar 2026 | Section 2.4      | 1.8     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 25 Mar 2026 | Section 2.5      | 1.9     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 27 Mar 2026 | Section 3.0      | 2.0     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 29 Mar 2026 | Section 3.1      | 2.1     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 30 Mar 2026 | Section 3.2, 3.3 | 2.2     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 02 Apr 2026 | Section 3.4      | 2.3     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 05 Apr 2026 | Section 3.5      | 2.4     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 30 Mar 2026 | Section 4.0      | 2.5     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 12 Apr 2026 | Section 4.1      | 2.6     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 16 Apr 2026 | Section 4.2, 4.3 | 2.7     |
| Maryam Shahzad             | Mr. Hasnain Tahir | 20 Apr 2026 | Section 4.4      | 2.8     |
| Muhammad Abdur Rehman Khan | Mr. Hasnain Tahir | 03 May 2026 | Section 5.0, 5.1 | 2.9     |
| Wasfa Nauman Bhatti        | Mr. Hasnain Tahir | 12 May 2026 | Section 5.2      | 3.0     |

# Contents

|                                                      |             |
|------------------------------------------------------|-------------|
| <b>Project Overview</b>                              | <b>ii</b>   |
| <b>Revision History</b>                              | <b>iv</b>   |
| <b>List of Figures</b>                               | <b>vii</b>  |
| <b>List of Tables</b>                                | <b>viii</b> |
| <b>1 Introduction</b>                                | <b>1</b>    |
| 1.1 Product Purpose . . . . .                        | 1           |
| 1.2 Product Scope . . . . .                          | 2           |
| 1.3 Objectives . . . . .                             | 5           |
| 1.4 Intended Market of Product . . . . .             | 6           |
| 1.5 Intended Users of the Product . . . . .          | 8           |
| <b>2 Background and Literature Review</b>            | <b>10</b>   |
| 2.1 Existing System Description . . . . .            | 10          |
| 2.2 Future System Usage Analysis . . . . .           | 22          |
| 2.3 Problem Statement / Limitations . . . . .        | 23          |
| 2.4 Proposed Solution . . . . .                      | 23          |
| 2.5 Software Process Model . . . . .                 | 24          |
| 2.5.1 Introduction . . . . .                         | 25          |
| 2.5.2 Justification . . . . .                        | 25          |
| 2.5.3 Steps . . . . .                                | 26          |
| <b>3 Software Requirements Specification</b>         | <b>30</b>   |
| 3.1 Introduction . . . . .                           | 30          |
| 3.1.1 Document Scope . . . . .                       | 30          |
| 3.1.2 Audience . . . . .                             | 31          |
| 3.2 Functional Requirements . . . . .                | 31          |
| 3.3 Non-Functional Requirements . . . . .            | 32          |
| 3.3.1 Software Quality Attributes . . . . .          | 33          |
| 3.3.2 Performance Requirements . . . . .             | 33          |
| 3.3.3 Safety Requirements . . . . .                  | 34          |
| 3.3.4 Other Non-Functional Requirements . . . . .    | 34          |
| 3.4 Requirements Gathering Techniques Used . . . . . | 35          |
| 3.4.1 Interviews . . . . .                           | 35          |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 3.4.2    | Questionnaire . . . . .               | 36        |
| 3.5      | Time Frame . . . . .                  | 37        |
| <b>4</b> | <b>Software Design Specification</b>  | <b>39</b> |
| 4.1      | Entity-Relationship Diagram . . . . . | 39        |
| 4.2      | Use-Case Diagram . . . . .            | 40        |
| 4.3      | Use-Case Descriptions . . . . .       | 41        |
| 4.4      | Sequence Diagrams . . . . .           | 56        |
| <b>5</b> | <b>Interfaces and Physical Design</b> | <b>64</b> |
| 5.1      | User Interfaces . . . . .             | 64        |
| 5.2      | User Tables . . . . .                 | 72        |
|          | <b>References</b>                     | <b>84</b> |

# List of Figures

|      |                                                    |    |
|------|----------------------------------------------------|----|
| 2.1  | Agile Process Model . . . . .                      | 25 |
| 4.1  | ER Diagram of the Proposed System . . . . .        | 40 |
| 4.2  | Use-Case Diagram of the Proposed System . . . . .  | 41 |
| 4.3  | Sequence Diagram for Register . . . . .            | 57 |
| 4.4  | Sequence Diagram for Login . . . . .               | 57 |
| 4.5  | Sequence Diagram for Recover Password . . . . .    | 58 |
| 4.6  | Sequence Diagram for Create Organization . . . . . | 58 |
| 4.7  | Sequence Diagram for Generate Data . . . . .       | 59 |
| 4.8  | Sequence Diagram for Simulate Resources . . . . .  | 59 |
| 4.9  | Sequence Diagram for View Resources . . . . .      | 60 |
| 4.10 | Sequence Diagram for Detect Threats . . . . .      | 60 |
| 4.11 | Sequence Diagram for Create Policy . . . . .       | 61 |
| 4.12 | Sequence Diagram for Execute Remediation . . . . . | 61 |
| 4.13 | Sequence Diagram for Forecast Costs . . . . .      | 62 |
| 4.14 | Sequence Diagram for Generate Reports . . . . .    | 62 |
| 4.15 | Sequence Diagram for Manage Settings . . . . .     | 63 |
| 5.1  | Sign Up Screen . . . . .                           | 65 |
| 5.2  | Account Login Screen . . . . .                     | 65 |
| 5.3  | Recover Password Screen . . . . .                  | 66 |
| 5.4  | Dashboard Screen . . . . .                         | 66 |
| 5.5  | Organization Manager Screen . . . . .              | 67 |
| 5.6  | Membership Management Screen . . . . .             | 67 |
| 5.7  | Data Generator Screen . . . . .                    | 68 |
| 5.8  | Resource Simulator Screen . . . . .                | 68 |
| 5.9  | Resource Viewer Screen . . . . .                   | 69 |
| 5.10 | Threat Detection Screen . . . . .                  | 69 |
| 5.11 | Policy Manager Screen . . . . .                    | 70 |
| 5.12 | Cost Forecast Screen . . . . .                     | 70 |
| 5.13 | Report Generator Screen . . . . .                  | 71 |
| 5.14 | Learning Lab Screen . . . . .                      | 71 |
| 5.15 | Settings Screen . . . . .                          | 72 |

# List of Tables

|      |                                   |    |
|------|-----------------------------------|----|
| 2.1  | Applications Comparison . . . . . | 22 |
| 3.1  | Functional Requirements . . . . . | 32 |
| 3.2  | Time Frame . . . . .              | 37 |
| 5.1  | User . . . . .                    | 72 |
| 5.2  | Settings . . . . .                | 73 |
| 5.3  | PasswordReset . . . . .           | 73 |
| 5.4  | EmailVerification . . . . .       | 74 |
| 5.5  | Notification . . . . .            | 74 |
| 5.6  | AuditTrail . . . . .              | 75 |
| 5.7  | UserRole . . . . .                | 75 |
| 5.8  | Organization . . . . .            | 75 |
| 5.9  | Invitation . . . . .              | 76 |
| 5.10 | Membership . . . . .              | 76 |
| 5.11 | BaselineProfile . . . . .         | 77 |
| 5.12 | SimulationRun . . . . .           | 77 |
| 5.13 | CostForecast . . . . .            | 78 |
| 5.14 | Budget . . . . .                  | 78 |
| 5.15 | Policy . . . . .                  | 78 |
| 5.16 | SimulatedResource . . . . .       | 79 |
| 5.17 | PolicyViolation . . . . .         | 79 |
| 5.18 | TrafficLog . . . . .              | 80 |
| 5.19 | ResourceMetric . . . . .          | 80 |
| 5.20 | ThreatLog . . . . .               | 81 |
| 5.21 | Alert . . . . .                   | 81 |
| 5.22 | RemediationAction . . . . .       | 82 |
| 5.23 | LearningLab . . . . .             | 82 |
| 5.24 | Report . . . . .                  | 83 |



# Chapter 1

## Introduction

The Cloud Policy, Cost and Security Simulator is an innovative web-based Digital Twin cloud simulation platform that helps students, cloud learners, and beginner IT professionals understand cloud governance, financial operations, resource monitoring, and cybersecurity workflows in a safe and controlled environment. The system helps users gain practical exposure through simulated virtual machines, databases, organizations, policies, reports, dashboards, labs, and security activities rather than relying on costly production cloud environments. The system enables users to create simulated infrastructure, observe real-time metrics, analyze cloud-resource behavior, manage governance rules, view cost summaries, and understand operational risks through interactive dashboards. The platform incorporates synthetic telemetry, organization-based isolation, simulated threat detection, reporting, learning labs, profile management, and settings workflows to make cloud education more practical and engaging. The system helps bridge the gap between theoretical cloud-computing education and practical operational experience, where students often face difficulty accessing real cloud platforms due to cost, complexity, and fear of misconfiguration. The system creates a reusable learning environment where cloud understanding depends on experimentation, guided interaction, and visualization-based practice.

### 1.1 Product Purpose

The proposed system functions as a Digital Twin cloud simulation and educational platform that enables students, cloud learners, and beginner IT professionals to practice cloud governance, cost management, resource monitoring, and security workflows through simulated cloud environments. Its primary purpose is to provide practical cloud learning exposure for users who understand cloud concepts theoretically but lack access to safe operational environments due to financial limitations, billing risks, infrastructure complexity, and security concerns associated with production cloud platforms. The system operates through a controlled web-based environment where users can create organizations, manage simulated virtual machines and databases, observe real-time dashboards, analyze operational metrics, configure governance policies, review reports, monitor simulated threats, and understand cloud-resource behavior without connecting to real cloud providers. Synthetic telemetry, organization-based isolation, reporting workflows, learning labs, profile management, and cost-visualization features improve operational understanding by transforming abstract cloud concepts into practical and repeatable ed-

educational workflows. Through these integrated features, the platform aligns with the broader goal of improving cloud literacy, operational confidence, cybersecurity awareness, and practical infrastructure-management understanding within academic and technical learning environments while maintaining safe and reusable experimentation for all users [1].

## 1.2 Product Scope

The product scope extends across the domains of cloud simulation, governance management, resource monitoring, security awareness, cost visualization, organization handling, reporting, and learning-oriented cloud practice. It functions as a web-based Digital Twin environment where users interact with simulated cloud infrastructure and understand operational behavior through dashboards, reports, policies, labs, metrics, and security summaries. Its purpose is to address the lack of safe and affordable cloud-practice environments for students and beginner professionals. By combining synthetic metric generation, simulated resource lifecycle management, organization-based isolation, governance validation, cost summaries, and threat-monitoring workflows, it establishes a complete educational platform for practical cloud learning. The system is required because traditional cloud learning often depends on videos, theory, limited labs, or paid platforms that do not allow continuous experimentation. The platform resolves such a problem by offering reusable cloud simulations, visual operational feedback, and educational workflows that help learners understand cloud-resource behavior. Its importance lies in connecting theoretical knowledge with hands-on practice, improving cloud literacy, and supporting safe experimentation in academic and training environments. Research shows that simulation-based learning environments improve practical understanding, engagement, and skill development in technical education [2].

The module defines key functional elements that shape the system's structure, including authentication, organization management, synthetic data generation, resource simulation, dashboard monitoring, security detection, governance validation, cost management, reporting, labs, membership handling, and settings. These components ensure safe access, organized cloud workspaces, simulated infrastructure behavior, real-time visibility, educational engagement, and practical cloud-management understanding within the Digital Twin environment.

### **Module: Authentication**

- User registration
- User login
- Recover password

- Email verification
- Profile access

**Module: Organization Manager**

- Create organization
- Manage members
- Assign roles
- Accept invitation
- Tenant isolation

**Module: Data Generator**

- Generate metrics
- Generate traffic
- Create baselines
- Build histories
- Support simulations

**Module: Resource Simulator**

- Create virtual machines
- Create databases
- Manage lifecycle
- Track runtime
- Simulate infrastructure

**Module: Resource Viewer**

- View resources
- Monitor metrics
- Display charts
- Show topology
- Receive updates

**Module: Threat Detector**

- Analyze traffic

- Detect threats
- Show alerts
- Classify activity
- Summarize risks

**Module: Policy Engine**

- Create policies
- Validate compliance
- Track violations
- Score governance
- View audit activity

**Module: Cost Forecaster**

- View cost trends
- Create budgets
- Show summaries
- Detect wastage
- Display forecasts

**Module: Report Generator**

- Generate reports
- Export PDF
- Export CSV
- Summarize costs
- Summarize security

**Module: Remediation Agent**

- Automated response
- Mitigation workflow
- Alert handling
- Recovery guidance
- Threat containment

**Module: Audit Trail**

- Activity logging
- Change history
- Event tracking
- User actions
- Timeline records

**Module: User Settings**

- Edit profile
- Change theme
- Manage preferences
- Configure alerts
- Update account

### 1.3 Objectives

Objectives in research or software engineering work consist of clear, specific, measurable, and achievable statements that define intended accomplishments, development targets, project scope, and evaluation criteria. These statements transform a broad system idea into practical milestones that guide design, implementation, testing, documentation, and final assessment. Such objectives are essential because they direct the development process, support resource allocation, maintain stakeholder alignment, and prevent uncontrolled expansion of the project scope. Strong objectives help developers make consistent decisions throughout execution and provide a verifiable foundation for confirming project success. Without well-defined objectives, development efforts may become unclear, incomplete, or misaligned with user needs. Precise objectives improve communication among team members, strengthen academic documentation, support risk management, and allow reviewers to evaluate system relevance accurately. They also help prioritize features, support iterative refinement, and maintain traceability from requirements to implementation. Effective objectives follow frameworks such as SMART criteria, which encourage accountability and systematic delivery in software projects [3].

The system aims to achieve a set of specific objectives that establish a practical, safe, and educational platform for cloud-computing learners. Its primary objective is to enable students, cloud trainees, and beginner IT professionals to practice cloud operations through simulated infrastructure instead of production environments. The platform allows users to create organizations, manage virtual machines and databases, monitor metrics, configure policies, analyze costs, detect simulated threats, and generate reports within a controlled Digital Twin environment. Another objective is to integrate synthetic

metric generation so users can observe realistic resource behavior such as CPU usage, memory load, network traffic, storage utilization, and operational state changes. The system also aims to promote security awareness through simulated threat detection and alert workflows. A further objective is to support educational engagement through cloud labs, scenario-based learning, profile management, and progress-oriented interaction. Through these combined objectives, the system improves practical understanding, cloud literacy, and operational confidence.

The system will align its specific objectives with broader goals that emphasize accessibility, practical cloud education, operational readiness, and safe technical experimentation. It will ensure that learners without paid cloud subscriptions or enterprise access can still practice important cloud-management workflows. By providing dashboards, reports, governance modules, cost views, security summaries, and resource simulations, the platform will promote understanding of how cloud operations work in realistic environments. The integration of organization-based isolation will support learning about multi-tenant access and role-based management. The learning labs and scenario workflows will encourage users to practice repeatedly, observe outcomes, and improve technical confidence. The reporting and dashboard modules will support analysis, reflection, and academic demonstration. Through these objectives, the system will contribute toward building an educational cloud simulation environment where users develop practical skills through interaction, visualization, and controlled experimentation rather than passive theory alone.

## 1.4 Intended Market of Product

The proposed Cloud Policy, Cost and Security Simulator is intended for multiple educational and technical markets that require practical cloud-learning and infrastructure simulation environments. The following market descriptions explain the major target markets of the proposed system along with the ways in which the platform addresses their operational and educational requirements. Each market section highlights the characteristics, challenges, and expected benefits associated with the adoption of the proposed platform.

**Higher Educational Institutions:** Higher educational institutions include universities, colleges, and academic departments that offer courses related to cloud computing, cybersecurity, software engineering, and information technology. These institutions currently require practical educational platforms where students can understand infrastructure behavior, governance workflows, cloud-resource management, and operational monitoring beyond theoretical lectures and static demonstrations. Many institutions face limitations because production cloud platforms introduce subscription costs, infras-

structure complexity, operational risks, and restricted experimentation opportunities for beginner learners. The proposed Cloud Policy, Cost and Security Simulator is relevant to this market because it provides a reusable educational environment that supports cloud-resource simulation, dashboard monitoring, operational reporting, governance visibility, topology visualization, and security-oriented workflows through controlled educational interaction. The platform will allow higher educational institutions to improve practical cloud-learning activities while reducing dependency on expensive enterprise infrastructure. Students will gain better operational understanding through repeated experimentation, while instructors will receive safer and more structured environments for classroom demonstrations, guided labs, and educational cloud-training activities.

**Technical Training Institutes:** Technical training institutes represent organizations that focus on practical skill development, certification preparation, operational cloud training, and beginner technical education for aspiring IT professionals. These institutes currently require interactive systems where trainees can repeatedly practice cloud workflows, monitoring activities, governance visibility, and infrastructure analysis through safe experimentation. Existing training methods often depend on screenshots, temporary vendor labs, isolated tutorials, or restricted-access cloud sandboxes that limit operational learning and continuous interaction. The proposed simulator is suitable for this market because it combines cloud simulation, telemetry generation, reporting systems, threat monitoring, dashboard visibility, and governance-oriented educational workflows into one integrated platform. The implementation of the system will allow technical training institutes to improve learner engagement, strengthen practical cloud understanding, and provide reusable cloud-learning environments without requiring large operational budgets or enterprise subscriptions. Trainees will benefit from practical operational exposure, while institutes will receive safer and more flexible educational cloud-training infrastructure.

**Small and Medium Enterprises:** Small and medium enterprises represent organizations that require affordable cloud-learning and operational-awareness environments before transitioning toward enterprise cloud platforms and large-scale infrastructure management. These organizations currently face challenges in training employees because production cloud environments involve financial costs, operational responsibilities, security concerns, and infrastructure-management complexity that discourage experimentation. The proposed Cloud Policy, Cost and Security Simulator is relevant to this market because it provides controlled educational workflows that support governance visibility, resource monitoring, cloud-cost analysis, dashboard interaction, and simulated security activities without exposing organizations to production risks. The adoption of the proposed platform will help small and medium enterprises improve technical preparedness, operational awareness, and cloud-management understanding through safe experimen-

tion and guided simulation-oriented learning. Employees will develop stronger practical cloud skills, while organizations will reduce training costs and improve readiness for future cloud adoption and infrastructure-management activities.

## 1.5 Intended Users of the Product

The intended users consist of individuals and groups engaged in cloud education, computer science learning, cybersecurity training, and beginner-level infrastructure management who seek practical exposure to cloud operations. They include students pursuing technology-related degrees, cloud trainees preparing for industry roles, instructors teaching cloud-computing concepts, and beginner IT professionals building operational understanding. Each user seeks a platform that promotes safe cloud practice through simulation rather than direct exposure to costly production systems. The users are learning-oriented, technically curious, and motivated by practical skill development. They value systems that provide dashboards, reports, labs, resource controls, security summaries, cost views, and governance workflows in a clear environment. Many of them aim to improve their understanding of cloud operations through repeated experimentation and visual feedback. They also appreciate learning features that make complex cloud concepts easier to understand through scenarios, guided labs, and operational summaries. The system supports users who prioritize practical knowledge, safe learning, and technical confidence, offering them a space to practice cloud-resource management, governance, monitoring, and security workflows.

**Students:** Students will serve as the main learners who will use the platform to practice cloud-computing workflows, infrastructure management, governance validation, and security-awareness concepts. They will benefit from simulated virtual machines, databases, dashboards, cloud labs, reports, and cost summaries that help them understand how cloud systems behave. The platform will enable students to create resources, observe metrics, review operational activity, and understand the relationship between governance, security, cost, and infrastructure usage. Through scenario-based labs and visualization features, students will develop practical skills beyond theoretical classroom learning. The system will also help students build confidence by allowing repeated experimentation without fear of financial loss or production damage. By using the platform, students will gain exposure to real-world cloud-management patterns in a safe educational environment.

**Cloud Trainees:** Cloud trainees will utilize the platform to strengthen their understanding of operational cloud workflows before working with real infrastructure. They will interact with organizations, dashboards, simulated resources, policies, reports, and threat summaries to develop familiarity with practical cloud environments. The system



will allow trainees to explore resource lifecycle activities, metric patterns, budget summaries, governance violations, and security alerts through controlled simulations. This role will benefit from repeated learning exercises because the platform provides reusable environments that do not expire like many temporary cloud labs. Cloud trainees will also use reporting and visualization tools to analyze operational behavior and improve decision-making. Their involvement will support the platform's goal of preparing learners for cloud-related roles through safe and practical experimentation.

**Instructors:** Instructors will use the platform as a teaching and demonstration tool for explaining cloud governance, FinOps, monitoring, and security concepts. They will demonstrate how resources are created, how metrics change, how governance rules affect compliance, and how simulated threats appear in dashboards. The platform will help instructors connect theoretical lessons with practical visuals, making complex cloud concepts easier for students to understand. Instructors can use learning labs, reports, operational summaries, and dashboards during lectures or practical sessions. The simulator will reduce the need for paid cloud accounts during demonstrations, making it suitable for academic environments with limited budgets. Their use of the system will support interactive teaching, structured learning, and practical understanding across cloud-computing courses.

**Beginner IT Professionals:** Beginner IT professionals will use the system to improve operational awareness and prepare for cloud-related responsibilities. They will explore simulated cloud resources, organization workflows, dashboards, security alerts, cost views, and governance policies to understand how cloud systems are managed in professional environments. The platform will provide them with a safe space to experiment with infrastructure decisions and observe outcomes without affecting production systems. They will benefit from reports, labs, and visual summaries that support self-learning and skill development. By interacting with the simulator, these users will improve confidence in cloud-resource management, security monitoring, and governance workflows. Their role strengthens the platform's value as a bridge between academic learning and practical industry expectations.

# Chapter 2

## Background and Literature Review

A literature review analyzes and synthesizes previous studies, tools, and existing systems to identify established knowledge in a specific area. It serves as a foundation that guides new development by summarizing theories, methods, technologies, and outcomes from prior work. Through evaluating sources, it clarifies common patterns, limitations, contradictions, and innovations related to the proposed system. Researchers and developers use literature review to align their work with credible findings and justify the need for new systems. By comparing diverse platforms, a literature review strengthens the credibility of the study and refines the objectives of the proposed work [4][5][20].

The following sections describe existing cloud platforms, monitoring tools, simulation systems, and educational services related to the proposed project. They highlight the major functions, strengths, and limitations of each system. The chapter also presents future system usage analysis, problem statement, proposed solution, and software process model. The discussion collectively explains how the proposed Digital Twin platform will provide safe cloud simulation, governance practice, cost visualization, security-awareness workflows, and educational learning support.

### 2.1 Existing System Description

Studying existing systems provides clear understanding of how cloud platforms, monitoring tools, and simulation environments operate and support users. It enables developers to identify gaps, refine concepts, and apply proven strategies that enhance usability, performance, and interaction. By observing features, dashboards, governance tools, security workflows, and learning methods, the team gains guidance for creating an efficient and improved solution. Such evaluation promotes innovation, avoids repetitive structures, and ensures that new development meets real educational requirements. Reviewing established platforms strengthens decision-making by revealing approaches that support accessibility, monitoring, governance, and continuous learning. Analytical study encourages development of meaningful solutions that inspire practical cloud education, safe experimentation, and operational understanding within the technology community [5][20][21].

The review approach focuses on examining ten existing systems to identify their functions, strengths, and limitations. Each system is analyzed based on cloud features, mon-

itoring support, educational value, governance functions, security workflows, and technological design. The review explains how these systems operate and what challenges they present for students or beginner learners. Findings are organized in a structured format that highlights unique aspects of each system. A comparison table provides a clear evaluation, offering detailed insight into their performance and distinguishing elements.

**Amazon Web Services:** It operates as a large public cloud-computing platform that provides infrastructure, storage, databases, networking, monitoring, governance, and security services for organizations and developers. It allows users to create virtual machines through EC2, manage databases through RDS, monitor workloads through CloudWatch, and control permissions through IAM. The system is powerful for enterprise cloud deployment because it supports scalability, automation, global availability, and many service integrations. Its strength lies in providing real production cloud environments where organizations can run applications and manage operational workloads. Despite these advantages, AWS is difficult for beginner students because mistakes may cause billing charges, misconfigurations, and security risks. The platform also requires prior cloud knowledge, account setup, and continuous cost awareness. The proposed system differentiates itself by offering a risk-free local Digital Twin environment where students can practice cloud workflows without financial exposure [6].

The main features of the Application are listed below:

- **Cloud Infrastructure Services:** AWS provides compute, storage, database, networking, and security services for scalable application deployment.
- **Virtual Machine Provisioning:** AWS EC2 allows users to launch virtual machines with selected instance types, operating systems, storage, and network settings.
- **Monitoring Through CloudWatch:** AWS CloudWatch collects metrics, logs, and alarms to help users monitor workload performance and operational issues.
- **Identity and Access Management:** AWS IAM manages users, roles, policies, and permissions across cloud resources for access control.
- **Cost and Billing Tools:** AWS provides billing dashboards and cost-management tools to help track usage, spending, and resource expenses.

The limitations of the application are listed below:

- **Billing Risk:** Incorrect configuration or forgotten active resources may generate unexpected charges for beginner users.
- **High Complexity:** AWS contains many services and settings, making it difficult for beginners to understand resource relationships.

- **No Local Simulation Mode:** AWS does not provide a complete local simulator for practicing cloud workflows without production services.
- **Limited Beginner Guidance:** Real console workflows may overwhelm new users due to regions, instance types, permissions, and billing choices.
- **Production-Oriented Design:** AWS is mainly designed for enterprise workloads, making it less suitable for unrestricted classroom experimentation.

**Microsoft Azure:** It operates as a public cloud platform that provides compute services, storage, databases, monitoring, identity management, security, and governance tools for organizations. It allows users to create virtual machines, configure networks, monitor resources through Azure Monitor, and apply governance rules through Azure Policy. Azure is widely used in enterprise environments because it integrates with Microsoft services, supports hybrid infrastructure, and provides scalable cloud resources. Its strength lies in enterprise identity integration and structured cloud-management capabilities. Despite these advantages, Azure remains complex for students who are new to cloud computing. Creating resources, managing subscriptions, configuring monitoring, and understanding governance policies may be difficult without prior experience. Real usage can also involve cost and configuration risks. The proposed system addresses these issues by providing an educational simulator that teaches similar concepts through safe simulated resources, dashboards, policies, and cost views [7].

The main features of the Application are listed below:

- **Virtual Machine Management:** Azure allows users to create, resize, start, stop, and monitor virtual machines within cloud environments.
- **Azure Monitor:** Azure Monitor collects metrics, logs, and alerts to help organizations observe cloud-resource performance.
- **Azure Policy:** Azure Policy enables organizations to define rules and check compliance across cloud resources.
- **Identity Integration:** Azure integrates with Microsoft Entra ID for authentication, access control, and permission management.
- **Cost Management:** Azure provides cost dashboards and budget tools to help users track spending and resource usage.

The limitations of the application are listed below:

- **Subscription Dependency:** Azure requires account and subscription setup before users can fully experiment with cloud resources.

- **Configuration Difficulty:** Many services, resource groups, and access settings may confuse beginner cloud learners.
- **Real Cost Exposure:** Users may still face billing concerns when resources remain active or exceed free-tier limits.
- **Limited Academic Simulation:** Azure does not provide a local educational simulator combining governance, cost, security, and resource behavior.
- **Complex Governance Tools:** Azure Policy and monitoring tools are powerful but difficult for students needing simplified workflows.

**Google Cloud Platform:** It operates as a public cloud platform that provides virtual machines, storage, databases, monitoring, identity management, networking, and analytics services. Google Compute Engine allows users to create and run virtual machines on Google infrastructure, while Cloud Monitoring provides observability features for resources and applications. The platform is useful for scalable workloads, cloud-native applications, and data-driven systems. Its strength lies in performance, global infrastructure, and integration with analytics and machine-learning services. However, it still requires learners to understand service configuration, identity permissions, billing setup, and infrastructure-management practices. Beginner users may find it difficult to safely experiment with governance, cost tracking, and monitoring features because these activities happen in real cloud environments. The proposed system differs by simulating resource behavior, dashboards, cost summaries, governance checks, and security workflows inside a controlled educational platform [8].

The main features of the Application are listed below:

- **Compute Engine:** Google Cloud Compute Engine allows users to create and run virtual machines using selected configurations.
- **Cloud Monitoring:** Google Cloud Monitoring provides metrics, uptime checks, dashboards, and alerts for cloud resources.
- **Identity and Access Management:** Google Cloud IAM controls access through roles, permissions, and policies.
- **Cloud Console:** Google Cloud Console provides a central interface for managing resources, services, and operational activity.
- **Cost Management Tools:** Google Cloud provides billing reports and budgets to help users monitor usage and costs.

The limitations of the application are listed below:

- **Billing Configuration:** Google Cloud requires billing setup, which may discourage students from free experimentation.
- **Service Complexity:** Many services and configuration options make resource relationships difficult for beginners.
- **Limited Guided Simulation:** Google Cloud does not provide a complete educational simulator for safe cloud-management practice.
- **Risk of Misconfiguration:** Incorrect network, permission, or resource settings may create security or operational issues.
- **Advanced Learning Curve:** Monitoring, IAM, and compute settings require prior cloud understanding before effective use.

**VMware vSphere:** It operates as a virtualization platform that allows organizations to run, manage, and monitor virtual machines and infrastructure workloads. It supports datacenter virtualization, resource pooling, workload management, high availability, and enterprise infrastructure administration. VMware vSphere is widely used in organizations that manage on-premises or hybrid infrastructure environments. Its strength lies in virtual-machine control, infrastructure performance, and enterprise workload management. However, vSphere is not designed primarily for beginner cloud education. It usually requires access to virtualization servers, licenses, administrative knowledge, and infrastructure setup. Students may find it difficult to practice without institutional labs or dedicated hardware. The platform also focuses more on virtualization administration than integrated cloud-governance, FinOps, security simulation, and educational lab workflows. The proposed system provides a simpler web-based simulator where learners can practice cloud-resource behavior, dashboards, governance, cost summaries, and security workflows without enterprise virtualization infrastructure [9].

The main features of the Application are listed below:

- **Virtual Machine Management:** vSphere allows administrators to create, configure, migrate, and manage virtual machines.
- **Resource Pooling:** It allows compute, memory, storage, and network resources to be shared across virtual machines.
- **Infrastructure Monitoring:** vSphere provides monitoring and performance views for virtual infrastructure workloads.
- **High Availability Support:** vSphere supports workload resilience and high availability for enterprise environments.
- **Centralized Administration:** vCenter provides centralized management of hosts, clusters, and virtual machines.

The limitations of the application are listed below:

- **Hardware Dependency:** vSphere usually requires servers, virtualization hosts, or institutional lab environments.
- **Licensing Requirements:** Enterprise licenses and setup effort may limit access for students or small training centers.
- **Complex Administration:** Clusters, hosts, storage, and networking concepts may be difficult for beginner learners.
- **Limited Cloud Governance Focus:** vSphere focuses on virtualization rather than integrated cloud-governance simulation.
- **Not Designed for FinOps Learning:** It does not primarily teach cloud-cost forecasting, billing simulation, or budget summaries.

**Cisco Packet Tracer:** It operates as a network simulation platform used in educational environments to teach networking, IoT, and cybersecurity concepts. It allows students to design network topologies, configure routers and switches, simulate packet flows, and understand network behavior without physical hardware. Packet Tracer is useful for practical networking education because it gives students a safe virtual lab for experimentation. Its strength lies in visual simulation, beginner-friendly practice, and academic accessibility. However, Packet Tracer focuses mainly on networking rather than cloud infrastructure management. It does not provide simulated virtual machines, cloud databases, governance-policy validation, cost visualization, cloud dashboards, reporting systems, or organization-based tenant isolation. Students can learn networking concepts but cannot practice cloud FinOps, governance, operational monitoring, and Digital Twin cloud workflows through the same system. The proposed platform extends simulation-based learning toward cloud operations, cost management, security awareness, and governance practice [10].

The main features of the Application are listed below:

- **Network Topology Design:** Packet Tracer allows students to create network topologies without physical hardware.
- **Device Configuration:** Users can configure routers, switches, and end devices through graphical and command-line interfaces.
- **Packet Simulation:** The system shows packet movement across devices to support network-flow understanding.
- **Educational Labs:** Packet Tracer supports guided networking activities and classroom-based learning exercises.

- **Cybersecurity Practice:** The platform includes basic cybersecurity and IoT activities for introductory learning.

The limitations of the application are listed below:

- **No Cloud Resource Simulation:** Packet Tracer does not simulate virtual machines, databases, or cloud-service lifecycles.
- **Limited Governance Support:** It does not provide policy creation, compliance scoring, or cloud-organization workflows.
- **No FinOps Features:** Users cannot practice cost forecasting, budget tracking, or resource-optimization summaries.
- **Networking-Centric Scope:** The tool focuses on networking rather than integrated cloud dashboards and reports.
- **No Multi-Tenant Cloud Context:** It does not teach organization isolation, member roles, or cloud tenant workflows.

**AWS Educate:** It operates as an educational cloud-learning platform that provides students and educators with access to cloud-computing training, beginner resources, digital badges, and hands-on labs. The platform is designed to help learners understand cloud concepts and build skills using AWS-developed content. Its strength lies in structured training, beginner-friendly resources, and no-cost learning access for many users. AWS Educate introduces cloud topics through guided materials and selected hands-on activities. However, its learning environment remains connected to AWS services and predefined learning paths. Students may not receive a complete custom Digital Twin platform where governance, cost, security, reports, dashboards, profiles, organizations, and resource simulations are integrated together. The system supports cloud education but does not focus on building a local, reusable simulator where learners repeatedly experiment with cloud operations without service dependency. The proposed system offers a more project-specific simulation environment for continuous academic practice [11].

The main features of the Application are listed below:

- **Cloud Learning Content:** AWS Educate provides beginner-friendly resources that introduce cloud concepts and AWS services.
- **Hands-On Labs:** The platform offers selected labs that help students practice basic cloud skills.
- **Digital Badges:** Learners can earn badges after completing cloud-learning activities and training paths.



- **Career Pathways:** AWS Educate provides career-oriented learning paths for students preparing for cloud roles.
- **No-Cost Access:** Many learning resources are available without requiring a credit card.

The limitations of the application are listed below:

- **Platform Dependency:** AWS Educate focuses mainly on AWS-specific services and learning paths.
- **Limited Custom Simulation:** It does not provide a customizable local Digital Twin simulator for academic projects.
- **Predefined Learning Flow:** Fixed lessons reduce open-ended experimentation with simulated cloud environments.
- **Limited Module Integration:** Governance, FinOps, reports, threat detection, and dashboards are not combined in one simulator.
- **Less Control for FYP Demonstration:** Students cannot fully customize backend logic, dashboards, and database models.

**Azure Lab Services:** It operates as a cloud-based lab service that allows educators to create managed virtual-machine labs for students. It helps institutions provide practical computing environments without manually configuring physical lab machines. Students can access assigned virtual machines for coursework, development, or training activities. Its strength lies in simplifying classroom lab management and providing controlled virtual environments. However, Azure Lab Services focuses on providing temporary lab machines rather than teaching complete cloud governance, FinOps, security monitoring, dashboard visualization, and Digital Twin simulation workflows. It also depends on Azure subscriptions, educator setup, and cloud-hosted lab infrastructure. Students may practice inside virtual machines, but they do not necessarily learn how cloud resources, policies, dashboards, reports, simulated threats, and cost summaries interact. The proposed system differs by creating an educational cloud simulator where users actively manage simulated cloud resources and observe operational behavior across integrated modules [12].

The main features of the Application are listed below:

- **Managed Virtual Labs:** Azure Lab Services allows instructors to create managed virtual-machine labs for students.
- **Student Access Control:** Educators can assign lab access and manage student usage schedules.

- **Template-Based Setup:** Instructors can create reusable templates for student lab machines.
- **Cloud-Based Availability:** Students can access lab machines remotely through Azure infrastructure.
- **Classroom Management:** The platform helps educators manage lab access and virtual-machine usage.

The limitations of the application are listed below:

- **Subscription Dependency:** Azure Lab Services depends on Azure subscriptions and institutional setup.
- **Limited Cloud Governance Practice:** It provides lab machines but not detailed governance-rule simulation.
- **No Integrated Cost Simulation:** Students cannot practice budget tracking, wastage detection, or cost forecasting.
- **Temporary Lab Context:** Lab environments may be limited by schedules, quotas, or institutional rules.
- **Less Security Simulation:** It does not focus on threat detection, incident reports, or remediation workflows.

**Datadog:** It operates as a cloud monitoring and observability platform that helps teams view infrastructure health, application performance, logs, metrics, traces, alerts, and security-related information. It is used by DevOps teams, IT operations teams, and developers to monitor cloud and hybrid environments. Datadog provides dashboards, metric visualizations, monitors, alerting, and integrations with many technologies. Its strength lies in realtime observability and operational troubleshooting for production systems. However, Datadog is not designed as an educational cloud simulator for beginner learners. It requires integration with real infrastructure, agents, cloud accounts, containers, or services to collect operational data. Students may find it difficult to practice without existing infrastructure. The platform also does not provide a simulated environment where users can create virtual resources, generate synthetic metrics, practice governance rules, and view educational cost summaries. The proposed system offers these learning-focused features inside a controlled simulator [13].

The main features of the Application are listed below:

- **Infrastructure Monitoring:** Datadog visualizes hosts, containers, services, and infrastructure components.

- **Metrics and Dashboards:** The platform provides dashboards for metrics, distributions, and operational visualizations.
- **Alerts and Monitors:** Datadog monitors notify teams about failures, performance issues, or threshold changes.
- **Technology Integrations:** Datadog integrates with cloud providers, containers, databases, and applications.
- **Security and Log Visibility:** The platform includes logging and security-monitoring features for enterprise operations

The limitations of the application are listed below:

- **Requires Real Data Sources:** Datadog depends on connected infrastructure, agents, or cloud services.
- **Subscription Cost:** Advanced monitoring features may require paid plans, limiting academic accessibility.
- **Enterprise Complexity:** Many integrations and configurations may overwhelm beginner users.
- **No Resource Creation Simulation:** Datadog monitors existing systems but does not simulate cloud-resource creation.
- **Limited Educational Labs:** It does not provide guided learning overlays or beginner-focused simulation workflows.

**Splunk Observability Cloud:** It operates as an observability and monitoring platform that helps organizations analyze metrics, logs, traces, infrastructure behavior, application performance, and digital service reliability. Splunk supports operational teams by providing visibility across cloud-native and hybrid environments. The platform helps detect incidents, investigate root causes, and improve performance through correlated observability data. Its strength lies in enterprise-scale monitoring, troubleshooting, and analytics. However, Splunk Observability Cloud is not primarily designed for beginner cloud education or simulated infrastructure learning. It requires real applications, infrastructure, or service data to deliver meaningful insights. Students may struggle with setup, data ingestion, query concepts, and dashboard configuration. It also does not provide simple simulated cloud resources, governance-policy workflows, FinOps summaries, and cloud labs inside one academic platform. The proposed system differs by providing an educational simulator where operational behavior is generated and visualized for learning purposes [14].

The main features of the Application are listed below:

- **Unified Observability:** Splunk combines metrics, logs, traces, and performance data for operational visibility.
- **Infrastructure Monitoring:** It helps monitor cloud and hybrid infrastructure components across connected environments.
- **Real-Time Dashboards:** The system provides dashboards for service reliability and performance visualization.
- **Incident Troubleshooting:** Splunk supports root-cause analysis and incident investigation using correlated data.
- **Enterprise Integration:** Splunk integrates with several enterprise systems and operational data sources.

The limitations of the application are listed below:

- **Not a Simulator:** Splunk analyzes real operational data instead of simulating cloud resources.
- **Setup Complexity:** Data ingestion, dashboards, and alerting rules require technical experience.
- **Cost Barriers:** Enterprise observability tools may involve subscription costs for academic users.
- **Limited Cloud-Governance Practice:** It does not primarily teach policy validation or simulated governance enforcement.
- **No Integrated Learning Labs:** Splunk does not combine labs, cost simulation, resource creation, and governance workflows.

**OpenStack:** It operates as an open-source cloud infrastructure platform that controls compute, storage, and networking resources throughout a datacenter. It allows administrators and users to provision virtual machines, manage networks, use dashboards, and operate private-cloud environments. OpenStack is powerful because it provides cloud infrastructure components that can be deployed in on-premises or public-cloud contexts. Its strength lies in open-source flexibility, production-scale infrastructure management, and community-driven development. However, OpenStack deployment is complex and usually requires servers, networking knowledge, storage configuration, and administrative expertise. Beginner students may struggle to install, configure, and operate OpenStack without institutional support. The platform also focuses on real infrastructure rather than simplified educational Digital Twin simulation. The proposed system differs by providing a lightweight web-based simulation where students can learn cloud-resource behavior, governance workflows, cost visualization, and security awareness without deploying an actual private cloud [15].

The main features of the Application are listed below:

- **Open-Source Cloud Infrastructure:** OpenStack provides open-source components for compute, storage, and networking.
- **Dashboard-Based Provisioning:** OpenStack includes dashboards for provisioning and managing cloud resources.
- **Compute Resource Management:** OpenStack manages virtual machines and compute resources through its Nova component.
- **Network and Storage Services:** The platform provides networking and storage services for cloud environment
- **Community Development:** OpenStack is supported by a large open-source cloud community.

The limitations of the application are listed below:

- **Difficult Deployment:** OpenStack installation requires servers, networking setup, storage planning, and technical knowledge.
- **Hardware and Infrastructure Needs:** Practical usage often requires multiple machines or a strong lab environment.
- **Advanced Learning Curve:** Beginners may struggle with its many services, APIs, and configuration layers.
- **No Simplified Educational Flow:** OpenStack is designed for real infrastructure rather than guided classroom simulation.
- **Limited FinOps Simulation:** It does not focus on beginner-friendly cost forecasting or synthetic billing workflows.

Table 2.1 compares the distinct advantages offered by the proposed system when measured against conventional cloud platforms, monitoring tools, educational services, and infrastructure-management systems. It provides a clear view of how the proposed system combines simulation, governance, security monitoring, cost visualization, reporting, and educational learning within one integrated Digital Twin platform.

Table 2.1: Applications Comparison

| Features                   | Applications            |                     |                           |                    |                          |                  |                         |              |                                 |                |                 |
|----------------------------|-------------------------|---------------------|---------------------------|--------------------|--------------------------|------------------|-------------------------|--------------|---------------------------------|----------------|-----------------|
|                            | Amazon Web Services [6] | Microsoft Azure [7] | Google Cloud Platform [8] | VMware vSphere [9] | Cisco Packet Tracer [10] | AWS Educate [11] | Azure Lab Services [12] | Datadog [13] | Splunk Observability Cloud [14] | OpenStack [15] | Proposed System |
| User Profile Management    | ✗                       | ✗                   | ✗                         | ✗                  | ✗                        | ✓                | ✓                       | ✗            | ✗                               | ✗              | ✓               |
| Organization Workspace     | ✓                       | ✓                   | ✓                         | ✓                  | ✗                        | ✗                | ✓                       | ✓            | ✓                               | ✓              | ✓               |
| VM and Database Simulation | ✗                       | ✗                   | ✗                         | ✗                  | ✗                        | ✗                | ✗                       | ✗            | ✗                               | ✗              | ✓               |
| Realtime Dashboard         | ✓                       | ✓                   | ✓                         | ✓                  | ✓                        | ✗                | ✗                       | ✓            | ✓                               | ✓              | ✓               |
| Governance Policy Module   | ✓                       | ✓                   | ✓                         | ✗                  | ✗                        | ✗                | ✗                       | ✗            | ✗                               | ✓              | ✓               |
| Cost Visualization         | ✓                       | ✓                   | ✓                         | ✗                  | ✗                        | ✗                | ✗                       | ✓            | ✗                               | ✗              | ✓               |
| Threat Detection View      | ✓                       | ✓                   | ✓                         | ✗                  | ✓                        | ✗                | ✗                       | ✓            | ✓                               | ✗              | ✓               |
| Learning Labs              | ✗                       | ✗                   | ✗                         | ✗                  | ✓                        | ✓                | ✓                       | ✗            | ✗                               | ✗              | ✓               |
| Report Export              | ✓                       | ✓                   | ✓                         | ✓                  | ✗                        | ✗                | ✗                       | ✓            | ✓                               | ✗              | ✓               |
| Risk-Free Simulation       | ✗                       | ✗                   | ✗                         | ✗                  | ✓                        | ✓                | ✓                       | ✗            | ✗                               | ✗              | ✓               |

## 2.2 Future System Usage Analysis

The platform system will function as a practical educational hub where students, cloud trainees, instructors, and beginner IT professionals can learn cloud operations through safe Digital Twin simulations. Users will create organizations, provision simulated virtual machines and databases, observe real time dashboards, analyze cost summaries, configure governance policies, and complete cloud-lab scenarios. The system will support future academic usage by helping instructors demonstrate resource behavior, governance validation, security alerts, and operational monitoring without real cloud infrastructure. As the system evolves, it may include more advanced analytics, multi-cloud simulation patterns, improved learning recommendations, expanded reporting, and stronger visualization of infrastructure topology. It may also support additional attack scenarios, advanced cost forecasting, better progress tracking, and richer role-based learning. Through continuous improvement, the platform will support cloud education, cybersecurity awareness, and FinOps learning in academic institutions. It will empower learners by making cloud operations accessible, reusable, and safe while maintaining practical relevance for modern IT environments [3][5][20][21].

## 2.3 Problem Statement / Limitations

The system addresses the ongoing challenge faced by students and beginner IT learners who need practical cloud-management experience but cannot easily access safe production-like environments. Real cloud platforms require subscriptions, billing configuration, access control, security awareness, and infrastructure knowledge, which creates barriers for learners. Students may accidentally create costly resources, expose services, misconfigure permissions, or fail to understand resource behavior due to lack of guided practice. Existing educational platforms often provide either theoretical material, temporary labs, or service-specific exercises that do not combine governance, cost, security, resource simulation, reporting, and learning workflows in one environment. This limits practical understanding and reduces confidence among learners entering cloud-related fields [5][24].

The proposed system also faces certain limitations that influence its design and development process. Since the platform is a simulator, it does not connect to real AWS, Azure, or Google Cloud accounts and cannot replace production cloud-management tools. The generated metrics, threats, and costs are simulated for educational purposes, so they represent learning behavior rather than real enterprise telemetry. Time constraints may limit the depth of AI models, the number of supported cloud-resource types, and the complexity of advanced remediation workflows. Maintaining accurate simulation logic, smooth real-time dashboards, and useful learning scenarios also requires continuous testing and refinement. These limitations are addressed by clearly positioning the system as an educational Digital Twin simulator and by focusing on safe, understandable, and practical workflows.

## 2.4 Proposed Solution

This section presents the proposed solution designed to address the problems and limitations identified in the previous section. The proposed solution introduces a web-based Digital Twin cloud simulation platform named Cloud Policy, Cost and Security Simulator that promotes practical cloud learning through safe infrastructure simulation, governance validation, cost visualization, security monitoring, reporting, and educational workflows. It provides a structured environment where students, cloud trainees, instructors, and beginner IT professionals can create organizations, manage simulated virtual machines and databases, monitor dashboards, analyze resource behavior, and understand cloud operations without using real cloud providers. The platform integrates synthetic telemetry, governance modules, threat-detection workflows, reporting systems, and FinOps-oriented cost summaries to improve operational understanding and educational engagement. Dataset-driven simulation using Alibaba Cluster Trace v2018 and CICIDS2017 subsets helps generate realistic infrastructure metrics, workload activity, and simulated

threat behavior for learning purposes. Governance modules allow users to create policies, monitor compliance, and observe operational violations, while security modules generate alerts and risk summaries that improve cloud-security awareness. Cost-management features provide budget views, wastage detection, and forecasting summaries that teach resource-optimization concepts without financial risk. Reporting, audit logs, and learning labs support guided experimentation and operational review within a safe educational environment. The solution addresses the limitations of existing cloud platforms by removing billing dependency, reducing operational complexity, and combining governance, monitoring, security, simulation, and reporting within one integrated system. By merging educational simulation, dashboards, policy management, cost analysis, and security workflows, the platform establishes a practical and reusable cloud-learning environment that supports accessibility, operational confidence, and technical skill development. The subsequent chapters further discuss the requirements, design structure, and implementation details of the proposed system [1][2][24].

## 2.5 Software Process Model

A software process model defines the structured framework that guides system development to ensure efficiency, quality, maintainability, and organized execution. It describes the stages involved in software creation, including planning, analysis, design, implementation, testing, deployment, and evaluation. Using a structured process model helps developers manage tasks systematically while maintaining communication, consistency, and development quality throughout the project life cycle. The model also supports risk management, requirement tracking, and continuous validation to ensure that project objectives remain aligned with user expectations. A suitable process model improves productivity, reduces development errors, supports collaboration among team members, and provides a road map for transforming system requirements into a reliable and user-centered solution [16][20][21].

The Agile Iterative Model is selected for the development of the proposed system because it supports flexibility, continuous improvement, collaboration, and adaptive development workflows. The model divides development activities into smaller iterations that allow the team to implement modules gradually while receiving regular testing feedback and evaluation. This approach suits the proposed Digital Twin cloud simulation platform because the system contains multiple interconnected modules such as authentication, organization management, resource simulation, governance validation, dashboards, threat detection, reporting, and cost management. The iterative structure supports progressive enhancement, frequent testing, and modular development while allowing changes and improvements throughout implementation. The model also encourages user-centered development and efficient management of evolving project requirements.



### 2.5.1 Introduction

The Agile Iterative Model is selected as the software process model for the development of the proposed Cloud Policy, Cost and Security Simulator because it supports flexibility, continuous improvement, modular implementation, and iterative development workflows throughout the project life cycle. The model divides development activities into smaller iterations where planning, design, implementation, testing, and evaluation are performed repeatedly to improve functionality and software quality after each cycle. This approach is highly suitable for the proposed Digital Twin cloud simulation platform because the system contains multiple interconnected modules such as Authentication, Organization Manager, Resource Simulator, Threat Detector, Policy Engine, Report Generator, and Cost Forecaster that require gradual implementation and continuous refinement. The Agile Iterative Model allows the development team to test modules individually, identify issues early, adapt to requirement changes, and improve system reliability through repeated evaluation and feedback. The model also promotes collaboration, adaptive planning, progressive enhancement, and efficient management of evolving project requirements throughout development. Continuous testing and iterative feedback further improve usability, maintainability, scalability, and operational consistency within the proposed platform. Research studies indicate that Agile-based software development improves flexibility, quality assurance, iterative delivery, and user-centered system design in modern software-engineering projects [16]. Figure 2.1 illustrates the workflow of the Agile Iterative Model used during the development of the proposed system.

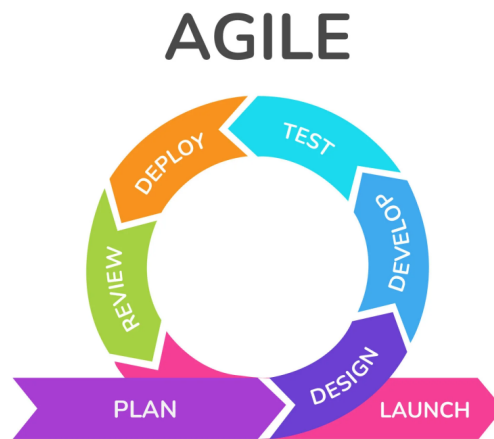


Figure 2.1: Agile Process Model [16]

### 2.5.2 Justification

The Agile Iterative Model will be chosen because it will provide flexibility, adaptability, and consistent improvement throughout development. It will enable the team to divide tasks into shorter iterations, allowing each cycle to produce a functional and

testable version of the system. Regular feedback from users, supervisor reviews, and implementation testing will guide refinements, ensuring that outcomes remain aligned with expectations. The model will support collaboration by encouraging open communication, frequent evaluations, and active participation from all team members. Risks will be minimized through early detection of issues, reducing delays and improving reliability. Productivity will increase as the model will structure work into achievable milestones that maintain steady progress. Its adaptability will allow integration of new modules such as dashboards, labs, reports, and governance workflows without disrupting development. Continuous testing and validation will maintain quality and prevent performance issues. The model will also ensure scalability, enabling the system to evolve with changing requirements. By emphasizing responsiveness, teamwork, and iterative enhancement, the Agile Iterative Model will ensure efficient delivery of a reliable, user-focused cloud simulation platform [16][17].

### 2.5.3 Steps

The development process follows the Agile Iterative Model, emphasizing flexibility, collaboration, and continuous enhancement. Each iteration includes planning, design, development, testing, and evaluation to ensure progressive improvement. The model supports adaptive workflows, enabling efficient development and timely delivery of a reliable, user-centered Digital Twin cloud simulation platform.

**Sprint 1 (Authentication Module):** In the first sprint, the development team will focus on implementing the authentication module to establish secure access control. The backend team will configure APIs required for user registration, login, password recovery, email verification, token handling, and profile access. The frontend team will design login, registration, forgot-password, reset-password, verification, and profile interfaces with suitable validation. This sprint will emphasize user identity, secure password handling, and session management. Testing will confirm that users can register, log in, recover passwords, and access protected routes before using other platform modules.

**Sprint 2 (Organization Manager Module):** In the second sprint, the development team will focus on implementing the organization-management module to support tenant isolation and workspace management. The backend team will configure APIs required for organization creation, organization listing, membership management, role assignment, invitation handling, and organization switching. The frontend team will design organization pages, invitation interfaces, membership panels, and workspace-selection controls with suitable validation and access restrictions. This sprint will emphasize organization isolation, member-role control, and secure workspace separation. Testing will confirm that users can create organizations, manage members, assign roles, accept invitations,

and access organization-specific resources without cross-organization data exposure.

**Sprint 3 (Data Generator Module):** In the third sprint, the development team will focus on implementing the data-generation module to support realistic cloud simulation workflows. The backend team will configure services required for generating synthetic telemetry, workload metrics, operational histories, cost signals, and simulated traffic activity. The frontend team will design metric views, charts, summaries, and monitoring components for displaying generated operational data. This sprint will emphasize realistic infrastructure behavior, organization-linked metric generation, and dynamic dashboard support. Testing will confirm that generated values remain stable, realistic, and properly connected with simulated cloud resources.

**Sprint 4 (Resource Simulator Module):** In the fourth sprint, the development team will focus on implementing the resource-simulation module to support virtual infrastructure management. The backend team will configure APIs required for virtual-machine creation, database creation, lifecycle actions, status updates, and runtime metric tracking. The frontend team will design resource pages, creation forms, lifecycle controls, and infrastructure-management interfaces with suitable validation. This sprint will emphasize realistic cloud-resource behavior, organization-linked simulation, and infrastructure lifecycle management. Testing will confirm that users can create, start, stop, restart, and monitor simulated resources accurately before integration with monitoring and governance modules.

**Sprint 5 (Resource Viewer Module):** In the fifth sprint, the development team will focus on implementing the dashboard and resource-viewer module to improve infrastructure visibility and operational monitoring. The backend team will configure APIs required for dashboard summaries, operational metrics, topology data, trend analysis, and realtime monitoring values. The frontend team will design dashboard cards, charts, topology views, resource tables, and monitoring panels for displaying infrastructure activity visually. This sprint will emphasize dashboard accuracy, realtime updates, and organization-specific data visualization. Testing will confirm that monitoring views, charts, summaries, and topology components display simulated infrastructure behavior correctly.

**Sprint 6 (Threat Detector Module):** In the sixth sprint, the development team will focus on implementing the threat-detection module to support simulated security monitoring and alert generation. The backend team will configure logic required for analyzing suspicious traffic, identifying simulated attacks, classifying threat activity, and generating alerts with severity levels and confidence values. The frontend team will design security dashboards, alert panels, threat summaries, and risk-indicator interfaces for displaying security-related activity. This sprint will emphasize security awareness,

alert visibility, and simulated incident monitoring. Testing will confirm that simulated threats are classified accurately and displayed consistently across the platform.

**Sprint 7 (Policy Engine Module):** In the seventh sprint, the development team will focus on implementing the governance and policy-management module to support compliance monitoring and rule validation. The backend team will configure APIs required for policy creation, compliance scoring, governance summaries, violation tracking, and rule validation workflows. The frontend team will design policy forms, governance dashboards, compliance views, and violation tables with suitable validation mechanisms. This sprint will emphasize governance visibility, organizational compliance, and policy-based resource evaluation. Testing will confirm that policies are created correctly, violations are detected accurately, and compliance results remain synchronized with simulated resources.

**Sprint 8 (Cost Forecaster Module):** In the eighth sprint, the development team will focus on implementing the cost-management and FinOps module to improve cloud-cost awareness and spending analysis. The backend team will configure services required for cost calculations, budget summaries, forecasting outputs, wastage detection, and trend generation. The frontend team will design cost dashboards, budget panels, optimization summaries, and financial charts for displaying simulated spending behavior. This sprint will emphasize FinOps visibility, budget awareness, and resource-cost relationships. Testing will confirm that cost summaries, charts, forecasting outputs, and budget thresholds display accurate simulated financial activity.

**Sprint 9 (Report Generator Module):** In the ninth sprint, the development team will focus on implementing the reporting module to support operational analysis and academic documentation workflows. The backend team will configure APIs required for generating security summaries, governance reports, cost summaries, resource-activity reports, and exportable data outputs. The frontend team will design report pages, export controls, printable summaries, and report-filtering interfaces for viewing generated results. This sprint will emphasize report accuracy, structured documentation, and organization-level filtering. Testing will confirm that generated reports, exports, summaries, and downloadable files function correctly across the platform.

**Sprint 10 (Remediation Agent Module):** In the tenth sprint, the development team will focus on implementing the remediation and response-management module to support simulated recovery and mitigation workflows. The backend team will configure logic required for mitigation suggestions, recovery recommendations, incident handling, and automated response workflows for simulated threats. The frontend team will design remediation panels, recovery interfaces, mitigation summaries, and response-management views with suitable workflow controls. This sprint will emphasize operational recovery,

incident awareness, and security-response understanding. Testing will confirm that remediation suggestions, mitigation workflows, and recovery summaries remain synchronized with threat-detection results.

**Sprint 11 (Audit Trail Module):** In the eleventh sprint, the development team will focus on implementing the audit-trail and activity-logging module to improve operational transparency and event tracking. The backend team will configure APIs required for activity logging, event recording, timeline generation, change tracking, and operational-history management. The frontend team will design audit dashboards, activity feeds, searchable timelines, and history panels for displaying recorded system activity. This sprint will emphasize accountability, operational visibility, and organization-specific event tracking. Testing will confirm that logged activities, timelines, event histories, and audit records remain accurate and synchronized throughout the platform.

**Sprint 12 (User Settings Module):** In the twelfth sprint, the development team will focus on implementing the user-settings and profile-management module to improve personalization and account-management workflows. The backend team will configure APIs required for profile updates, notification settings, account preferences, theme management, and user-detail modification. The frontend team will design settings pages, profile forms, notification controls, and preference-management interfaces with suitable validation. This sprint will emphasize personalization, user experience, and account consistency throughout the platform. Testing will confirm that profile updates, preference changes, notification settings, and theme configurations function correctly across all modules.

# Chapter 3

## Software Requirements Specification

A Software Requirements Specification (SRS) defines the complete description of a software system's intended functionalities, constraints, and performance standards. It serves as a formal agreement between stakeholders, developers, and evaluators, ensuring mutual understanding of the system's goals and requirements. SRS provides a structured framework for design, development, testing, and maintenance, reducing ambiguity and monitoring risks. By detailing both functional and non-functional requirements, it establishes a foundation for quality assurance and system validation [17].

The subsequent sections of the Software Requirements Specification present a detailed exploration of the system's functional and non-functional requirements, user roles, system design expectations, and technological framework. Each section focuses on critical components such as authentication, organization management, synthetic data generation, resource simulation, dashboards, governance, cost management, threat detection, reports, labs, and settings workflows. These sections provide structured insight into how the platform ensures safe simulation, practical cloud learning, operational visibility, and user-centered interaction.

### 3.1 Introduction

The Software Requirements Specification will serve as a foundational document that will guide the development process, ensuring that every module and feature aligns with defined objectives and user needs. It will outline the system's structure, functionality, and performance expectations to maintain consistency during development. By defining clear requirements and standards, it will minimize ambiguity, enhance collaboration, and support efficient progress toward a successful and well-structured Digital Twincloud simulation platform [20][21].

#### 3.1.1 Document Scope

The Software Requirements Specification defines the complete functional and non-functional aspects of the system, including user interactions, performance standards, interface behavior, and operational constraints. It covers authentication, organization management, synthetic data generation, resource simulation, dashboards, threat detection, governance validation, cost management, reporting, labs, membership, settings, and profile-management features. External integrations with real cloud providers, physical

hardware management, and production cloud orchestration remain outside its boundaries, ensuring focused documentation.

### 3.1.2 Audience

The Software Requirements Specification is intended for developers, system analysts, testers, project supervisors, and stakeholders who participate in the system's development and evaluation. It serves as a reference to ensure shared understanding among all contributors. The document provides clarity on objectives, functionality, technical expectations, and quality standards, allowing each audience group to align their roles effectively for successful system implementation.

- **Developers:** Will obtain a detailed technical road map describing every module, interface, and workflow, enabling accurate coding and integration of system components. The document will serve as a reliable guide for maintaining consistency in functionality and ensuring that all modules meet intended performance and usability standards.
- **System Analysts:** Will utilize the document to interpret user requirements, confirm feasibility, and ensure that every specification aligns with the overall system objectives. It will help them maintain coherence between functional needs and technical design, supporting structured analysis and efficient planning.
- **Testers:** Will rely on the document to identify measurable requirements, define validation criteria, and create precise test cases for quality assurance. It will assist them in detecting inconsistencies early and ensuring that all functionalities perform as expected under defined constraints.
- **Project Supervisors:** Will use the document to review progress, verify academic alignment, and evaluate whether the implemented system follows the approved scope. It will support structured assessment of modules, requirements, and development consistency.
- **Stakeholders:** Will refer to the document to understand the system's purpose, expected outcomes, and operational boundaries. It will help them evaluate whether the final platform meets educational needs and project goals.

## 3.2 Functional Requirements

Functional Requirements define the specific behaviors, features, and operations that a software system must perform to fulfill user and project needs. They describe how the system responds to inputs, manages data, processes workflows, and produces expected outputs under defined conditions. These requirements play a vital role in ensuring that

the software functions as intended and meets its objectives effectively. By clearly specifying system actions and interactions, they provide a foundation for design, implementation, and testing. Functional Requirements establish measurable criteria that determine whether a system performs desired tasks accurately and reliably [17].

Table 3.1 outlines the Functional Requirements of the system, providing unique identifiers and concise descriptions of each function. It serves as a clear reference for developers, analysts, and testers to ensure all essential features and actions are defined and aligned with system objectives.

Table 3.1: Functional Requirements

| Requirement ID | Description                      |
|----------------|----------------------------------|
| FR1            | User can sign up.                |
| FR2            | User can sign in.                |
| FR3            | User can recover password.       |
| FR4            | User can verify email.           |
| FR5            | User can create organization.    |
| FR6            | User can manage profile.         |
| FR7            | User can create virtual machine. |
| FR8            | User can create database.        |
| FR9            | User can view dashboard.         |
| FR10           | User can generate report.        |
| FR11           | User can detect threats.         |
| FR12           | User can view alerts.            |
| FR13           | User can forecast cloud cost.    |
| FR14           | User can manage policies.        |
| FR15           | User can view audit trail.       |

### 3.3 Non-Functional Requirements

Non-Functional Requirements define the quality attributes and constraints that determine how a software system performs rather than what it does. They describe aspects such as reliability, usability, scalability, performance, security, and maintainability, ensuring that the system operates efficiently under various conditions. These requirements play a crucial role in maintaining stability, improving user satisfaction, and ensuring long-term sustainability. By establishing measurable performance standards, they guide developers in building robust and responsive systems that meet stakeholder expectations [19][20][21].

The following sub-sections present the key Non-Functional Requirements that define



the system's quality standards and operational constraints. They include performance efficiency, reliability, security, scalability, usability, maintainability, safety, and system compatibility. Each aspect explains how the system maintains stability, safeguards user data, supports simulated workloads, and delivers a smooth educational experience under different usage conditions.

### 3.3.1 Software Quality Attributes

Software Quality Attributes describe the essential characteristics that define the system's performance, reliability, and user satisfaction. They ensure that the software maintains efficiency, security, adaptability, and usability across diverse usage scenarios, forming the foundation for a stable and high-performing Digital Twin cloud simulation system [19].

- **Reliability:** The system should operate consistently without frequent failure, ensuring that authentication, organizations, simulations, dashboards, reports, and learning workflows remain available during normal educational usage.
- **Usability:** The interface should be intuitive and user-friendly, allowing students and beginner learners to navigate dashboards, resources, labs, policies, reports, and settings without unnecessary confusion.
- **Performance Efficiency:** The system should deliver quick responses, smooth dashboard loading, stable chart rendering, and efficient simulation updates while handling multiple workflows at the same time.
- **Security:** User data should be protected through secure authentication, token handling, access control, and organization isolation to prevent unauthorized access or misuse.
- **Scalability:** The system should be capable of supporting additional modules, resources, organizations, reports, labs, and simulation scenarios without major structural changes.
- **Maintainability:** The codebase should remain organized, modular, and documented clearly so debugging, updates, feature enhancements, and future integrations can be performed efficiently.

### 3.3.2 Performance Requirements

The system's performance should be optimized to ensure smooth and responsive user interactions across all modules. Page loading time should be minimal, and data retrieval operations should be processed efficiently to enhance user experience. Dashboard updates, metric visualization, report generation, and resource simulation should respond

within acceptable time limits during educational use. Server response time should remain consistent even when users create resources, switch organizations, view reports, or open labs. Database queries should be designed for rapid execution, supporting concurrent access without major performance drops. Real-time communication should remain stable, enabling smooth synchronization between dashboard state and simulated backend data. Resource utilization should be balanced to prevent overloading and maintain consistent throughput. The system should handle multiple simulated operations while maintaining accuracy and speed [19].

### 3.3.3 Safety Requirements

The system should be designed to prevent data loss, unauthorized access, accidental deletion, and inconsistent simulation behavior during critical operations. Backup-friendly structures and clear error-handling procedures should be implemented to ensure recovery in case of unexpected failures or disruptions. User sessions should be protected to prevent unauthorized actions, while organization isolation should ensure that users cannot access data from other workspaces. Since the platform is educational, simulated resources must remain separate from real cloud infrastructure to avoid financial, security, and operational risks. Dangerous actions such as deletion, role changes, or report generation should be handled through controlled workflows. Error messages should guide users without exposing sensitive technical details. These safety measures ensure that learners can experiment confidently while system integrity remains protected [19].

### 3.3.4 Other Non-Functional Requirements

Other Non-Functional Requirements include compatibility, maintainability, accessibility, portability, and documentation quality. The system should operate smoothly in modern web browsers and remain compatible with the selected frontend and backend technologies. It should support maintainable APIs, reusable components, modular services, and clear database structures so future improvements can be added easily. The user interface should remain accessible and understandable for students with limited cloud experience. The system should also support deployment in local or institutional environments for academic demonstration. Documentation should clearly describe setup steps, module responsibilities, system limitations, and testing expectations. These requirements ensure long-term usefulness, easier maintenance, improved academic evaluation, and better adoption in educational environments [19].

## 3.4 Requirements Gathering Techniques Used

Requirements gathering techniques are structured methods used to identify, analyze, and document the needs, expectations, constraints, and functional requirements of a software system. These techniques play a critical role in software development because they help ensure that the final product is aligned with user needs, technical limitations, and stakeholder expectations. Selecting appropriate techniques directly influences the accuracy, completeness, and quality of the requirements captured during the early phases of development. In a software project, effective requirements gathering reduces ambiguity, prevents unnecessary rework, improves communication, and provides a strong foundation for design, implementation, and testing. Therefore, requirement-gathering techniques shape the direction of the final product and support the development of a system that solves the intended problem effectively [17].

The selected requirement-gathering methods included interviews, questionnaires, observation of existing platforms, and document analysis. These methods supported better understanding of user expectations, platform limitations, academic requirements, and practical cloud-learning needs. They also helped identify which features should be prioritized, such as authentication, organization management, resource simulation, real-time dashboards, governance, cost management, threat detection, reports, labs, and profile settings. The gathered information guided system planning and ensured that the final platform addressed real educational gaps.

### 3.4.1 Interviews

Interviews are a direct requirements-gathering technique used to collect detailed information from stakeholders through structured or semi-structured discussions. This technique is valuable in software development because it allows the development team to understand user expectations, clarify unclear requirements, and explore practical problems from the stakeholder's perspective. Interviews are commonly conducted by preparing relevant questions, selecting suitable participants, and recording their responses for analysis. They help uncover hidden needs, operational challenges, and user preferences that may not be visible through documents or observation alone. As a result, interviews support the development of more user-centered and requirement-focused software systems [17].

For the proposed product, interviews were conducted with students, cloud learners, instructors, and beginner technical users to understand the limitations they faced while learning cloud-computing concepts. The discussions focused on practical access issues, dashboard understanding, governance confusion, billing fear, security-awareness gaps,

and the need for reusable simulation environments. Students were asked about their difficulties in practicing cloud-resource creation, monitoring, governance, and cost-related workflows. Instructors were asked about the need for visual demonstration tools during lectures and labs. Beginner technical users were asked about their experience with production cloud platforms and the complexity they faced while learning real cloud environments.

The interview results showed that learners understood cloud theory but lacked safe practical exposure to cloud-resource creation, dashboards, governance rules, cost behavior, and security workflows. Instructors highlighted the need for dashboards, reports, scenarios, and controlled workflows that could explain resource utilization, policy compliance, cost summaries, and security alerts clearly. Beginner technical users explained that production cloud platforms felt complex because of service variety, configuration options, billing rules, and security responsibilities. These findings helped clarify the need for a simplified Digital Twin simulator that supports repeated practice, visual learning, and practical cloud understanding without financial or production risk.

### 3.4.2 Questionnaire

A questionnaire is a requirements-gathering technique used to collect structured responses from participants through predefined questions. This technique is important in software development because it allows the development team to gather information from multiple users in a consistent and organized manner. Questionnaires may contain close-ended, open-ended, or scale-based questions depending on the type of information required. The technique helps identify common patterns, user expectations, repeated challenges, and preferred system features among the target audience. It is especially useful when feedback is required from a large group of users with different technical backgrounds and learning experiences [17].

For the proposed product, questionnaires were distributed among students, cloud learners, instructors, and beginner IT users to collect information regarding cloud-learning difficulties, educational limitations, and preferred platform features. The questionnaire included questions related to practical cloud-resource management, governance understanding, dashboard usability, monitoring visibility, cost-management concepts, and security-awareness workflows. Participants were asked whether they experienced difficulty understanding practical cloud-management activities through theoretical learning alone and whether a Digital Twin simulation platform would improve their practical understanding of cloud operations. The collected responses were reviewed to determine which features should receive greater priority during system planning and development.

The questionnaire responses confirmed that many learners preferred visual dash-

boards, guided workflows, scenario-based interaction, reports, and simulation-oriented activities instead of static documentation or tutorial-only learning approaches. Participants also highlighted the importance of governance visibility, monitoring systems, cost visualization, and security-awareness workflows within educational cloud environments. The collected feedback helped validate the inclusion of modules such as Resource Viewer, Resource Simulator, Threat Detector, Cost Forecaster, Policy Engine, Report Generator, and Audit Trail within the proposed platform. The questionnaire technique proved effective because it provided structured evidence that the proposed system should focus on safer experimentation, repeated practice, and simplified operational learning for beginner users.

The following are some of the questions included in the questionnaire:

- Do you face difficulty understanding practical cloud-resource management using only theoretical learning methods?
- Would a Digital Twin cloud simulation platform improve your understanding of governance, monitoring, and cost-management concepts?

### 3.5 Time Frame

The proposed system follows a structured development schedule consisting of requirement gathering, literature review, user-interface design, backend development, frontend implementation, simulation integration, testing, and documentation. Each stage contributes toward stable system development, organized execution, and successful delivery of the Digital Twin cloud simulation platform [16][21].

Table 3.2 presents the estimated time frame associated with major development activities. The timeline supports systematic progress, organized task distribution, and controlled completion of implementation, testing, and documentation phases.

Table 3.2: Time Frame

| Phase                             | Duration |
|-----------------------------------|----------|
| Sprint 1: Authentication          | 20 days  |
| Sprint 2: Organization Management | 20 days  |
| Sprint 3: Data Generator          | 20 days  |
| Sprint 4: Resource Viewer         | 20 days  |
| Sprint 5: Resource Simulator      | 20 days  |
| Sprint 6: Threat Detector         | 20 days  |

*Continued on next page*

Table 3.2 – *Continued from previous page*

| <b>Phase</b>                | <b>Duration</b> |
|-----------------------------|-----------------|
| Sprint 7: Report Generator  | 20 days         |
| Sprint 8: Policy Engine     | 20 days         |
| Sprint 9: Remediation Agent | 20 days         |
| Sprint 10: User Settings    | 20 days         |
| Sprint 11: Cost Forecaster  | 20 days         |
| Sprint 12: Audit Trail      | 20 days         |

# Chapter 4

## Software Design Specification

Software Design Specification (SDS) is a software-engineering document that describes the overall structure, components, database organization, interfaces, and interactions of a software system. It serves as a blueprint that guides developers during implementation by translating software requirements into technical design representations. SDS plays an important role in software-development projects because it helps improve communication among stakeholders, supports implementation planning, reduces development ambiguity, and ensures that system requirements are addressed systematically. It also assists developers in understanding how different modules interact with each other and how information flows throughout the system. A well-prepared design specification contributes to maintainability, scalability, consistency, and quality throughout the software-development lifecycle [20][21].

The following sections present the major design artifacts of the proposed Cloud Policy, Cost and Security Simulator. These sections include the Entity-Relationship Diagram, Use-Case Diagram, Use-Case Descriptions, and Sequence Diagrams. Together, these design models describe database structure, user interactions, system workflows, and communication among software components. They provide a comprehensive understanding of how the proposed system operates and support implementation activities during development.

### 4.1 Entity-Relationship Diagram

An Entity-Relationship Diagram (ERD) is a graphical representation used to model the logical structure of a database by illustrating entities, attributes, and relationships among system components. ERDs are widely used in software-development projects because they help developers understand how information is stored, organized, and connected throughout the system. They support database planning by reducing redundancy, improving consistency, and identifying relationships required for operational workflows. Proper database modeling also contributes to efficient data retrieval, simplified maintenance, and improved scalability. As a result, ERDs play an essential role in designing reliable and well-structured software systems [25][26][27].

Figure 4.1 illustrates the Entity-Relationship Diagram of the proposed system. The diagram highlights important entities such as User, UserRole, Organization, Membership,

SimulatedResource, ResourceMetric, Policy, PolicyViolation, ThreatLog, CostForecast, Report, Notification, LearningLab, AuditTrail, and Settings. It also represents the relationships between authentication workflows, organization management, cloud-resource simulation, governance validation, monitoring activities, reporting modules, and security-management functionalities. The representation supports communication among team members by simplifying database understanding and helping identify how system records remain connected during operational workflows.

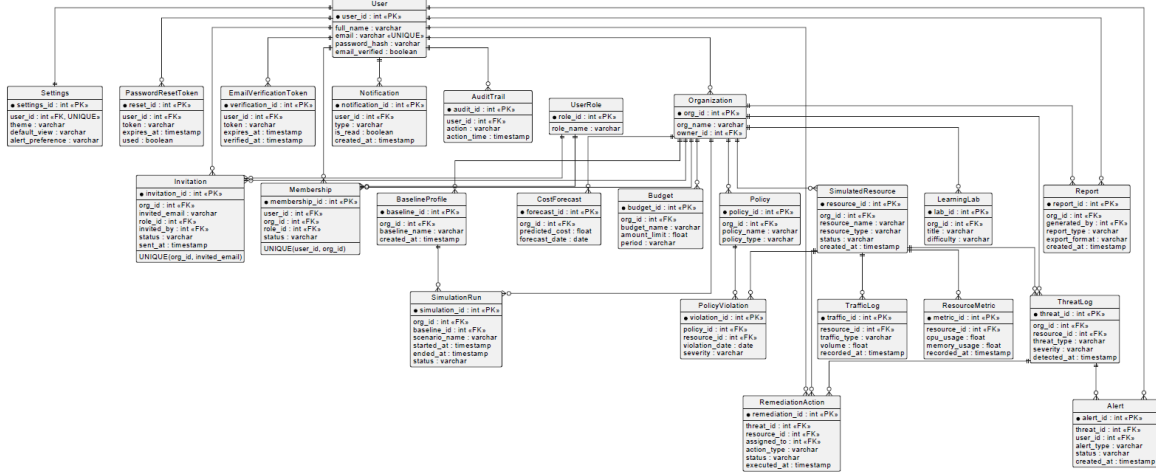


Figure 4.1: ER Diagram of the Proposed System

## 4.2 Use-Case Diagram

A Use Case Diagram is a behavioral UML diagram that represents the interaction between system users and software functionalities. It provides a high-level view of system behavior by identifying actors, use cases, and relationships between them. Use Case Diagrams are important in software-development projects because they help define system boundaries, clarify functional requirements, and improve communication among stakeholders. They also assist developers in understanding user expectations and operational workflows before implementation begins. By providing a visual representation of system functionality, use-case modeling contributes to requirement validation and software-planning activities [22][25].

Figure 4.2 illustrates the interactions between actors and system functionalities through a clear visual structure. The diagram highlights important actors including Student, Cloud Learner, Instructor, Beginner IT User, and Administrator. It also presents major system functionalities such as Register, Sign In, Recover Password, Create Organization, Manage Membership, Simulate Resources, View Dashboard, Detect Threats, Create Policies, Generate Reports, View Cost Forecasts, Access Learning Labs, and Manage Settings. The representation supports requirement understanding by simplifying functional expectations and helping identify key user-oriented behaviors that must be addressed during



implementation activities [22].

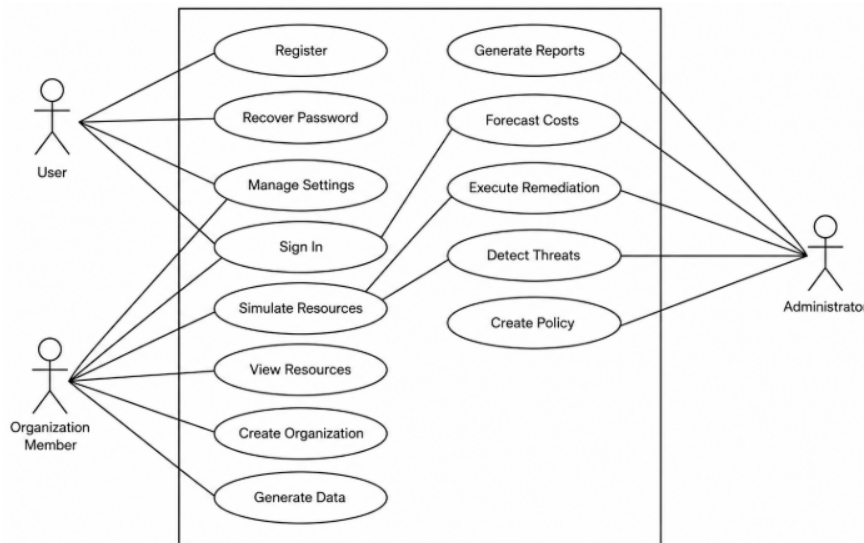


Figure 4.2: Use-Case Diagram of the Proposed System

## 4.3 Use-Case Descriptions

A Use Case Description provides a detailed explanation of how actors interact with a software system to perform specific operations. It expands the information presented in the Use Case Diagram by describing workflows, conditions, system responses, and exceptional situations associated with each functionality. Use Case Descriptions are important in software-development projects because they help developers understand operational requirements, support implementation planning, and assist in testing activities. They also improve communication among stakeholders by documenting how the system is expected to behave under different scenarios [20][22].

The following subsections provide detailed descriptions of the use cases identified in the Use Case Diagram. Each description follows the Rational Unified Process (RUP) format and includes the use-case identifier, actors, type, brief description, basic flow, alternative flow, pre-conditions, post-conditions, and exceptional scenarios associated with system operation.

### 4.3.1 UC01: Registration

**Actors:** User.

**Type:** primary.

**Brief Description:** The use case allows a new user to create an account within the proposed Cloud Policy, Cost and Security Simulator. The user provides personal and authentication information, which is validated before account creation. Successful registration

enables access to cloud-learning functionalities, dashboards, reports, and organization workspaces. The functionality serves as the initial entry point into the system.

**Basic Flow:**

1. The user opens the registration page.
2. The user enters registration information.
3. The system validates the provided details.
4. The system checks for duplicate accounts.
5. The system creates the user account.
6. The system displays a successful registration message.

**Alternative Flow:**

1. The user provides optional profile information.
2. The system stores additional profile details.
3. The user selects a different role during registration.
4. The system stores the selected role..

**Pre-condition:**

1. The user has access to the registration interface.
2. The user is not already registered in the system.

**Post-condition:**

1. A new user account is created successfully.
2. The user information is stored in the database.

**Exceptional Scenarios:**

1. Email address already exists.
2. Invalid information is entered.
3. Registration process fails.

### 4.3.2 UC02: User Login

**Actors:** User, Organization Member, Administrator.

**Type:** Primary.

**Brief Description:** The use case allows registered users to authenticate themselves and gain access to the system dashboard. The process verifies login credentials and grants access to authorized functionalities. Authentication protects organizational information and system resources from unauthorized access. Successful login enables users to perform cloud-learning and management activities.

**Basic Flow:**

1. The user opens the login page.
2. The user enters email and password.
3. The system validates credentials.
4. The system authenticates the user.
5. Dashboard is displayed.

**Alternative Flow:**

1. The user selects remember-me functionality.
2. The system stores session information.
3. Future logins are simplified.
4. Credentials are prefilled.

**Pre-condition:**

1. The user account already exists.
2. The user possesses valid login credentials.

**Post-condition:**

1. The user gains access to the dashboard.
2. The user is authenticated successfully.

**Exceptional Scenarios:**

1. Invalid credentials are entered.
2. Account is inactive.
3. Authentication fails.

### 4.3.3 UC03: Recover Password

**Actors:** User, Organization Member, Administrator.

**Type:** Primary.

**Brief Description:** The use case allows users to recover access to their accounts when passwords are forgotten. The system verifies identity through the registered email address and allows password modification. This functionality improves accessibility while maintaining account security. It ensures users can continue learning activities without administrator intervention.

**Basic Flow:**

1. The user selects forgot password.
2. The user enters email address.
3. The system verifies account information.
4. Password-reset process is initiated.
5. The user creates a new password.

**Alternative Flow:**

1. The user remembers password.
2. The user returns to login page.
3. The user requests another reset attempt.
4. The system restarts verification.

**Pre-condition:**

1. The user account exists.
2. The user has access to the registered email.

**Post-condition:**

1. Password is updated.
2. The user can log in using the new password.

**Exceptional Scenarios:**

1. Email address does not exist.
2. Verification process fails.

### 4.3.4 UC04: Create Organization

**Actors:** Organization Member.

**Type:** Primary.

**Brief Description:** The use case allows authenticated users to create organization workspaces within the simulator. Organizations provide isolated environments for managing resources, reports, policies, and learning activities. The functionality supports multi-tenant operation and collaborative learning workflows. It forms the foundation of cloud-management activities within the platform.

**Basic Flow:**

1. The user selects create organization.
2. The user enters organization details.
3. The system validates information.
4. Organization record is created.
5. Workspace becomes available.

**Alternative Flow:**

1. The user edits organization details before saving.
2. The user invites members during creation.

**Pre-condition:**

1. The user is authenticated.
2. The user has organization-creation permission.

**Post-condition:**

1. Organization is created.
2. The user becomes organization owner.

**Exceptional Scenarios:**

1. Organization name already exists.
2. Required information is missing..

### 4.3.5 UC05: Generate Data

**Actors:** Organization Member.

**Type:** Primary.

**Brief Description:** The use case allows the user to generate synthetic cloud-resource activity, metrics, workload histories, and traffic information for the proposed simulator. The generated data supports resource simulation, dashboard monitoring, cost forecasting, policy evaluation, and threat-analysis activities within the platform. The functionality uses processed datasets and simulation logic to imitate realistic cloud behavior in a controlled educational environment. It provides a foundation for practical experimentation by allowing users to observe how generated cloud activity affects system modules.

**Basic Flow:**

1. The user opens the Data Generator module.
2. The user selects the required generation option.
3. The user chooses workload type, traffic type, or metric profile.
4. The system validates the selected configuration.
5. The system generates synthetic cloud-resource data.
6. Generated data is stored in the database.
7. The system displays a success message to the user.

**Alternative Flow:**

1. The user selects a specific workload profile.
2. The system generates data according to the selected workload profile.
3. Generated workload data is saved for simulation and analysis.

**Pre-condition:**

1. The user is authenticated.
2. The organization workspace exists.

**Post-condition:**

1. Synthetic data is generated successfully. Generated data is stored and becomes available for analysis.

**Exceptional Scenarios:**

1. Required dataset is unavailable.
2. System displays a dataset-unavailable message.
3. Data generation process is stopped.

**4.3.6 UC06: Simulate Resources**

**Actors:** Organization Member.

**Type:** Primary.

**Brief Description:** The use case enables the user to create simulated virtual machines and databases within the learning environment. The simulator mimics cloud-resource behavior without connecting to real production cloud infrastructure. Users can observe resource status, utilization, runtime behavior, and operational changes through the system dashboard. This functionality supports practical understanding of cloud-resource management while maintaining a safe educational environment.

**Basic Flow:**

1. The user opens the Resource Simulator module.
2. The user selects the type of resource to simulate.
3. The user enters configuration details.
4. System validates the resource configuration.
5. System creates the simulated resource.
6. System stores resource information in the database.
7. System updates dashboard metrics and resource status.

**Alternative Flow:**

1. The user modifies resource configuration before creation.
2. System validates the updated configuration.
3. Resource is created using the modified settings.

**Pre-condition:**

1. The user is authenticated.
2. Organization workspace exists.

**Post-condition:**

1. Simulated resource is created successfully.
2. Resource information is stored and displayed in the system.

**Exceptional Scenarios:**

1. The user enters invalid resource configuration.
2. System displays a validation error.
3. Resource creation is stopped.

**4.3.7 UC07: View Resources**

**Actors:** Organization Member.

**Type:** Primary.

**Brief Description:** The use case allows the user to view simulated cloud-resource information through dashboards, monitoring screens, and resource-viewing interfaces. The system presents resource metrics, utilization summaries, operational status, and visual indicators in an understandable form. The functionality supports learning by helping users observe how cloud resources behave during simulated operations. It improves operational visibility and assists users in understanding cloud-resource management practices.

**Basic Flow:**

1. The user opens the Resource Viewer module.
2. System retrieves available resource records.
3. System retrieves related metric information.
4. System organizes resource data for display.
5. System presents resource status and utilization summaries.
6. The user reviews resource details through dashboards or charts.

**Alternative Flow:**

1. The user applies filters based on resource type or status.
2. System retrieves filtered records.
3. Filtered resource information is displayed.



**Pre-condition:**

1. The user is authenticated.
2. At least one simulated resource exists.

**Post-condition:**

1. Resource information is displayed to the user.
2. Dashboard statistics and visual summaries are updated.

**Exceptional Scenarios:**

1. Resource data is unavailable. System displays a no-data message.
2. Resource details are not shown.

### 4.3.8 UC08:Detect Threats

**Actors:** Administrator, Organization Member.

**Type:** Primary.

**Brief Description:** The use case allows the user to analyze simulated cloud activity and identify possible security threats within the learning environment. The Threat Detector evaluates resource metrics, generated traffic, and telemetry information to identify suspicious behavior. Detected threats are categorized according to severity and displayed through dashboards, summaries, and alerts. The functionality supports cybersecurity-awareness learning by showing how threat detection works in cloud-management workflows.

**Basic Flow:**

1. The user opens the Threat Detector module.
2. The user selects the analysis scope.
3. System retrieves available metrics and traffic data.
4. Threat-detection logic analyzes the data.
5. System classifies detected suspicious activity.
6. Threat records are stored in the database.
7. Results are displayed through alerts and summaries.

**Alternative Flow:**

1. The user applies custom filters before analysis.
2. System analyzes only the selected resources or time range.
3. Filtered threat results are displayed.

**Pre-condition:**

1. The user is authenticated.
2. Resource metrics or generated traffic data are available.

**Post-condition:**

1. Threat-detection results are generated.
2. Threat records are stored and displayed to the user.

**Exceptional Scenarios:**

1. Required metrics are unavailable.
2. System displays an analysis-unavailable message.
3. Threat detection is not performed.

### 4.3.9 UC09: Create Policy

**Actors:** Administrator.

**Type:** Primary.

**Brief Description:** The use case enables the user to define governance policies that regulate simulated cloud resources and operational behavior. Policies support compliance validation, governance scoring, and educational understanding of cloud-management rules. The functionality helps users understand how policy enforcement supports security, cost control, and resource governance in cloud environments. Created policies are stored and used for future evaluation against simulated resources.

**Basic Flow:**

1. The user opens the Policy Engine module.
2. The user selects create policy option.
3. The user enters policy name, rule type, and conditions.

4. System validates the policy definition.
5. System saves the policy record. Policy becomes available for evaluation.
6. System displays a success message

**Alternative Flow:**

1. The user modifies an existing policy.
2. System validates updated policy information.
3. Modified policy is saved.

**Pre-condition:**

1. The user is authenticated.
2. Organization workspace exists.

**Post-condition:**

1. Governance policy is stored successfully.
2. Policy becomes available for compliance evaluation.

**Exceptional Scenarios:**

1. The user enters an invalid policy definition.
2. System displays a validation error.
3. Policy is not saved.

**4.3.10 UC10: Execute Remediation**

**Actors:** Administrator.

**Type:** Primary.

**Brief Description:** The use case allows the user to apply corrective actions to identified governance violations or simulated security issues. The Remediation Agent provides recommended actions that help users understand how operational problems can be resolved. The functionality supports educational understanding of response handling, risk reduction, and cloud-governance correction workflows. Each remediation activity is recorded so users can review actions and outcomes later.

**Basic Flow:**

1. The user opens the Remediation Agent module.
2. System displays available threats or violations.
3. selects an issue for remediation.
4. System shows recommended corrective actions.
5. The user executes the selected remediation action.
6. System updates issue status.
7. activity is recorded in the audit trail.

**Alternative Flow:**

1. The user accepts the system recommendation.
2. Recommended remediation is applied.
3. Issue status is updated automatically.

**Pre-condition:**

1. A threat or policy violation exists.
2. The user has permission to perform remediation.

**Post-condition:**

1. Remediation action is recorded.
2. Issue status is updated in the system.

**Exceptional Scenarios:**

1. Remediation action fails during execution.
2. System displays an action-failure message.
3. Issue remains unresolved.

**4.3.11 UC11: Forecast Costs**

**Actors:** UserOrganization Member, Administrator.

**Type:** Primary.

**Brief Description:** The use case allows the user to estimate future cloud-resource expenses using simulated operational data. Cost forecasting helps users understand FinOps concepts, budget planning, resource utilization, and spending behavior in cloud environments. The module generates cost projections based on stored resource activity and simulated usage patterns. Results are displayed through dashboards and reports to support educational cost-management analysis.

**Basic Flow:**

1. The user opens the Cost Forecaster module.
2. The user selects forecasting option.
3. System retrieves resource usage data.
4. System calculates expected cost values.
5. Forecast results are generated.
6. System displays cost trends and summaries.
7. Forecast information is stored for reporting.

**Alternative Flow:**

1. The user changes the forecast duration.
2. System recalculates cost estimates.
3. Updated forecast results are displayed.

**Pre-condition:**

1. The user is authenticated.
2. Resource usage data is available.

**Post-condition:**

1. Cost forecast is generated successfully.
2. Forecasted cost information is displayed to the user.

**Exceptional Scenarios:**

1. Forecast data is unavailable.
2. System displays a no-data message.
3. Forecasting process is stopped.

### 4.3.12 UC12: Generate Reports

**Actors:** Organization Member, Administrator.

**Type:** primary.

**Brief Description:** The use case enables the user to generate reports related to governance, threats, resources, costs, and operational activity. Reports consolidate information from multiple modules and present summarized insights in an organized format. The functionality supports learning, evaluation, documentation, and review of simulated cloud operations. Generated reports help users analyze system behavior and export important information for academic or operational use.

**Basic Flow:**

1. The user opens the Report Generator module.
2. The user selects report type.
3. The user selects filters or report scope.
4. System retrieves required data.
5. System compiles report content.
6. Report is generated.
7. The user views or downloads the report.

**Alternative Flow:**

1. The user applies custom filters.
2. System generates report using selected filters.
3. Filtered report is displayed.

**Pre-condition:**

1. The user is authenticated.
2. Operational data is available in the system.

**Post-condition:**

1. Requested report is generated successfully.
2. Report becomes available for viewing or download.

**Exceptional Scenarios:**

1. No relevant data is available.
2. System displays a no-data message.
3. Report is not generated.

**4.3.13 UC13: Manage Settings**

**Actors:** User, Organization Member, Administrator.

**Type:** Primary.

**Brief Description:** The use case allows the user to customize profile information, notification preferences, interface settings, and account-related configurations. Personalization improves usability by allowing users to adapt the platform according to their learning and interaction needs. Updated settings are stored and applied across future sessions. The functionality supports better user experience and consistent system interaction.

**Basic Flow:**

1. The user opens the Settings module.
2. System displays current settings.
3. The user modifies profile or preference information.
4. System validates the changes.
5. Updated settings are saved.
6. System confirms successful update.
7. New preferences are applied.

**Alternative Flow:**

1. The user restores default settings.
2. System replaces current preferences with default values.
3. Default settings are saved.

**Pre-condition:**

1. The user is authenticated.
2. The user profile exists in the system.

**Post-condition:**

1. Settings are updated successfully.
2. New preferences are applied to future sessions.

**Exceptional Scenarios:**

1. The user provides invalid settings information.
2. System displays a validation error.
3. Settings are not updated.

## 4.4 Sequence Diagrams

A Sequence Diagram is a UML interaction diagram that represents how actors, system components, interfaces, and databases communicate during the execution of a specific function. It shows the order of messages exchanged between objects and explains how a process moves from request to response. Sequence diagrams are important in software-development projects because they help developers understand runtime behavior, system logic, and communication flow between modules. They also support implementation planning by showing how user actions are handled by the system and how data is stored or retrieved from the database. Through these diagrams, the development team can identify process dependencies, validation steps, and interaction patterns before implementation begins [22][25].

Sequence diagrams provide a detailed representation of system behavior by illustrating how different components interact during the execution of a specific task. They help developers understand the sequence of operations, data exchange, and control flow among users, interfaces, services, and databases. By presenting interactions in a chronological manner, sequence diagrams support validation of functional requirements and improve communication among development team members. They also assist in identifying dependencies between modules and ensuring that system processes are implemented according to the intended design specifications [22][25].

The following sequence diagrams are presented against the use cases described in the previous section. Each sequence diagram matches a relevant use-case description and explains the communication flow between the user, system module, service layer, and database. These diagrams help illustrate how authentication, organization management, data generation, resource simulation, monitoring, threat detection, policy creation, remediation, forecasting, reporting, and settings workflows operate within the proposed Cloud Policy, Cost and Security Simulator.



Figure 4.3 shows how a user registers an account. The user submits details to the system, which stores them and confirms successful registration back to the user.

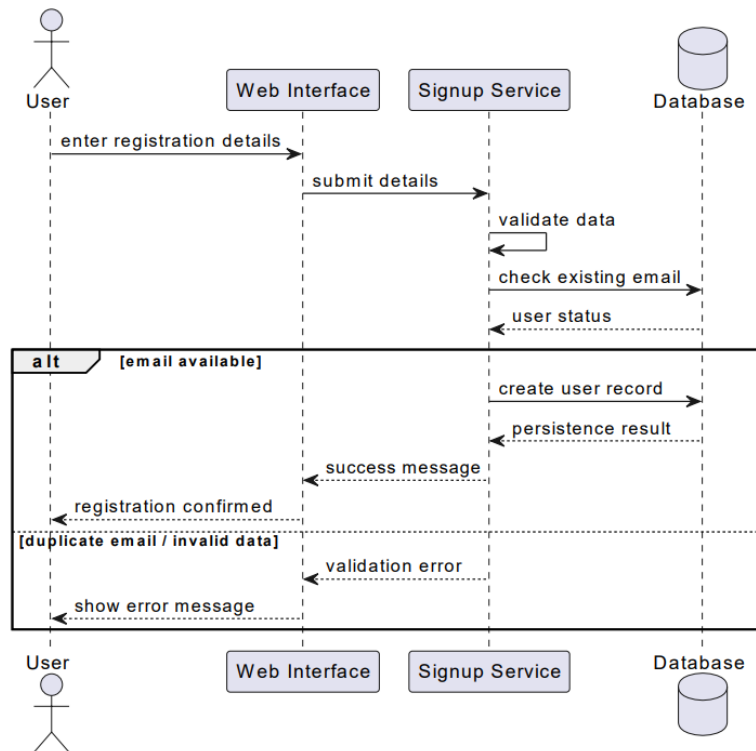


Figure 4.3: Sequence Diagram for Register

Figure 4.4 illustrates how users log in with email and password. The authentication system checks credentials in the database, validates them, and then grants or denies access accordingly.

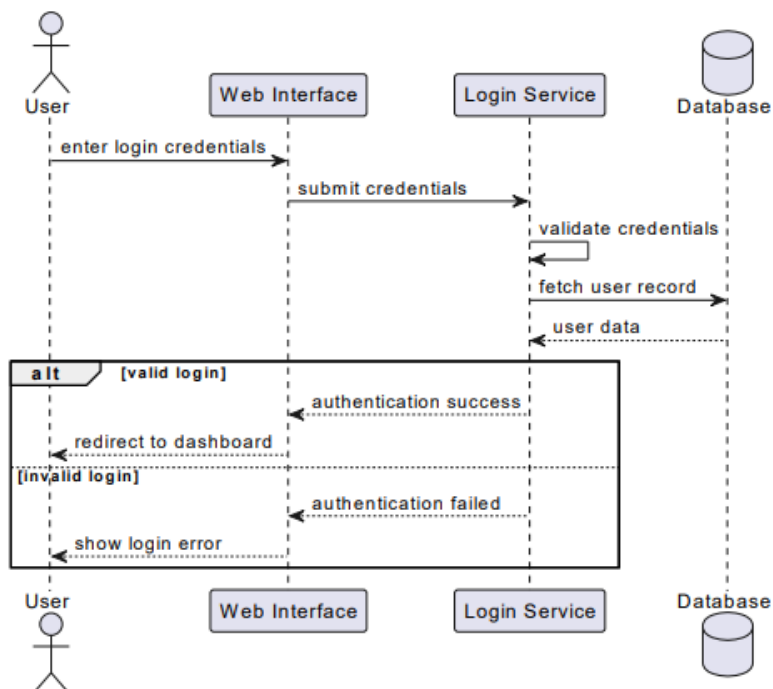


Figure 4.4: Sequence Diagram for Login

Figure 4.5 demonstrates the password-recovery process. The user submits a registered email address, the system verifies the account, generates and stores password-reset token, and provides reset instructions to the user.

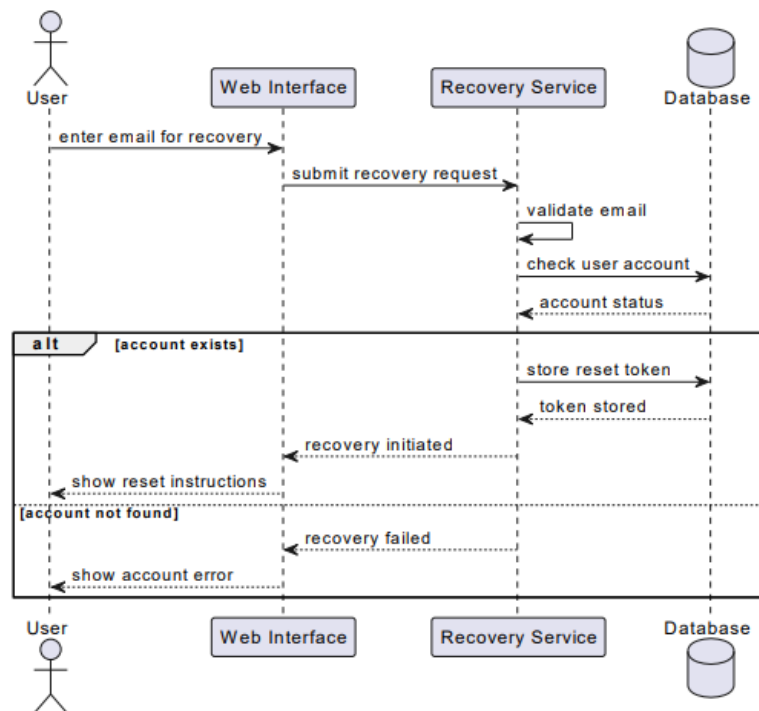


Figure 4.5: Sequence Diagram for Recover Password

Figure 4.6 shows how an organization workspace is created. The Organization Service validates organization details, stores the organization record, creates ownership membership information, and confirms successful creation

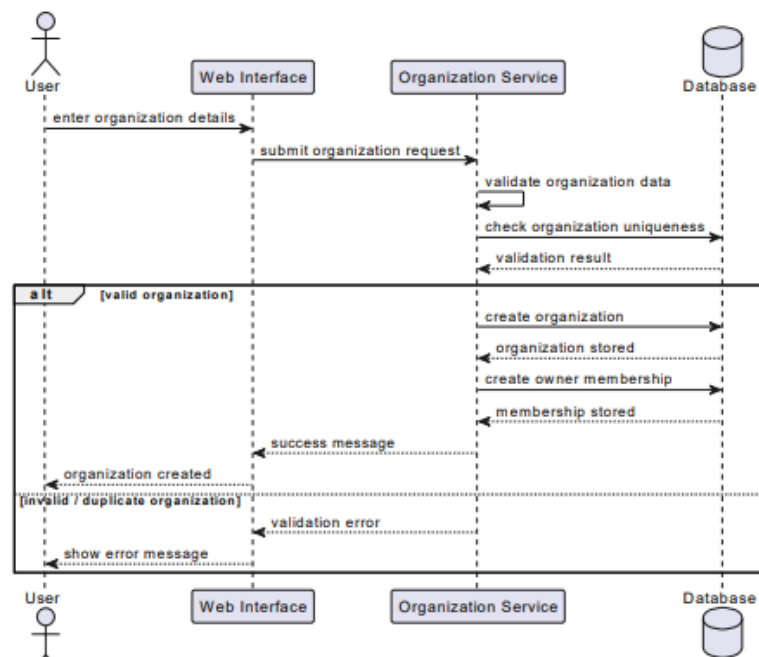


Figure 4.6: Sequence Diagram for Create Organization

Figure 4.7 illustrates the synthetic-data generation process. The Data Generator Service validates generation parameters, creates synthetic cloud data, stores the generated information, and displays the result to the user.

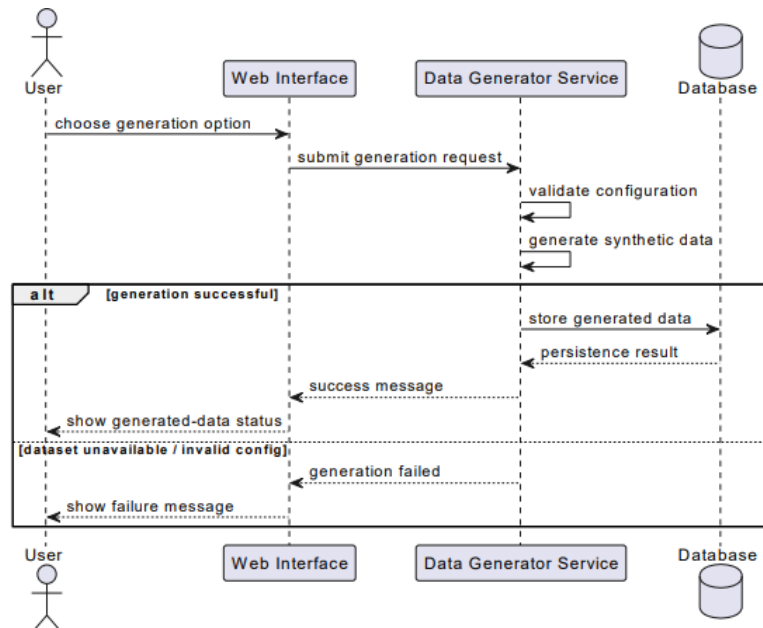


Figure 4.7: Sequence Diagram for Generate Data

Figure 4.8 demonstrates how simulated resources are created. The Resource Simulator Service validates resource configurations, creates simulated cloud resources, updates metrics, and displays resource status information.

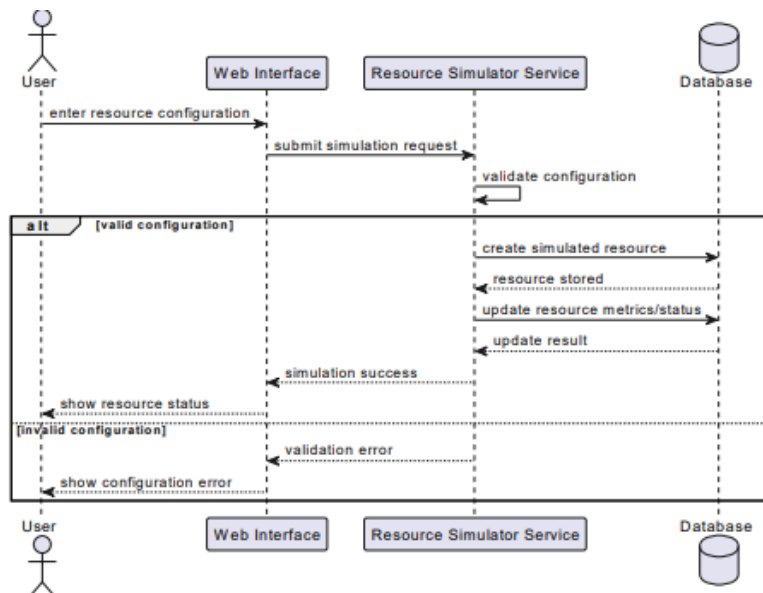


Figure 4.8: Sequence Diagram for Simulate Resources

Figure 4.9 shows how resource information is viewed within the platform. The Resource Viewer retrieves resource records and monitoring metrics from the database before presenting them through dashboards and visual summaries.

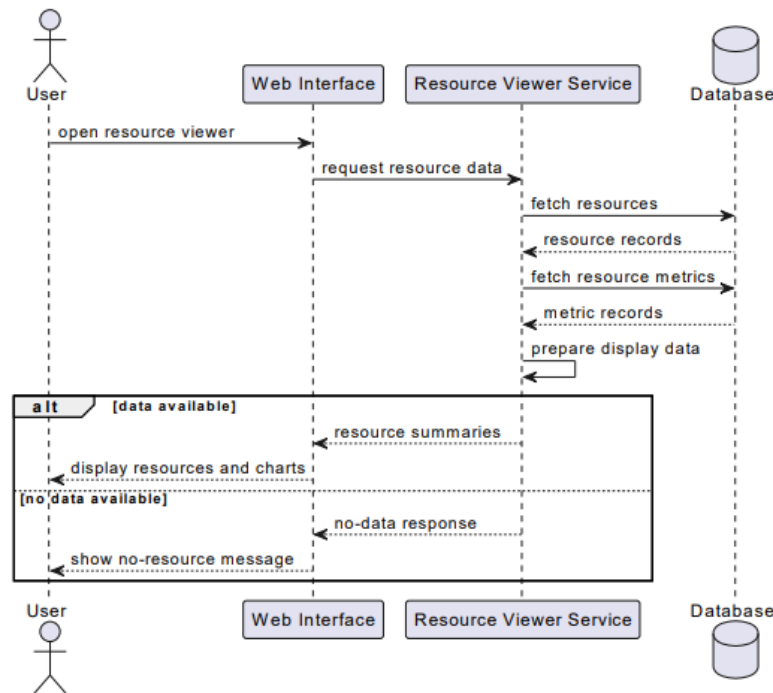


Figure 4.9: Sequence Diagram for View Resources

Figure 4.10 illustrates the threat-detection process. The Threat Detector analyzes telemetry information, identifies suspicious activities, stores threat records, and displays alerts and threat summaries to the user.

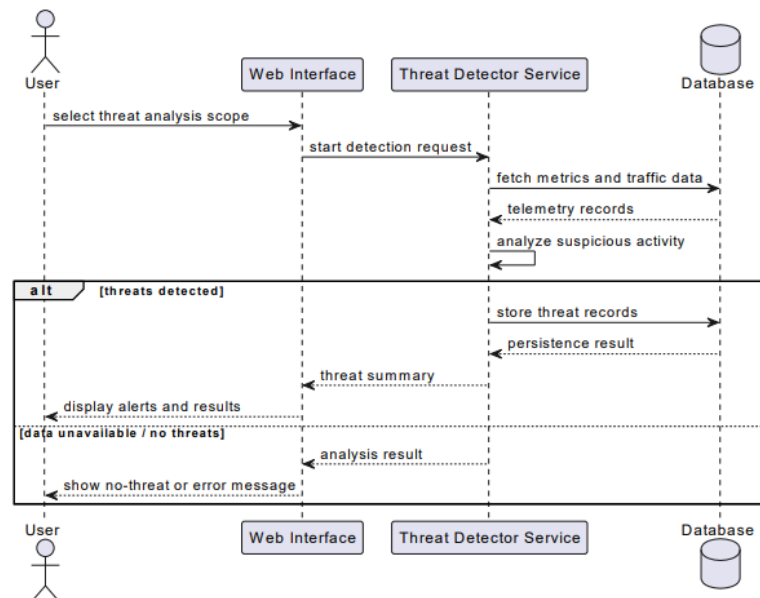


Figure 4.10: Sequence Diagram for Detect Threats

Figure 4.11 demonstrates how governance policies are created. The Policy Service validates policy definitions, stores policy information in the database, and confirms successful policy creation.

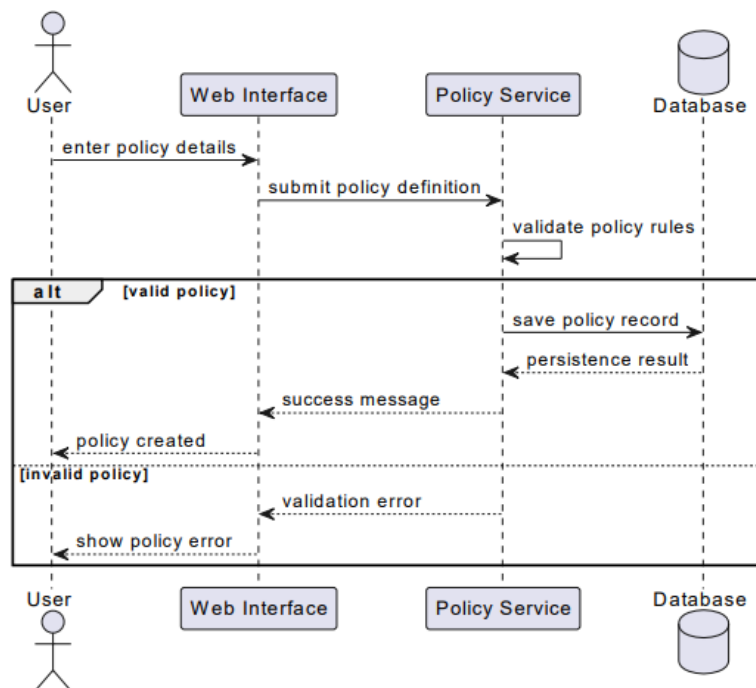


Figure 4.11: Sequence Diagram for Create Policy

Figure 4.12 shows how remediation activities are executed. The Remediation Service retrieves issue information, performs corrective actions, updates issue status records, and displays the remediation outcome.

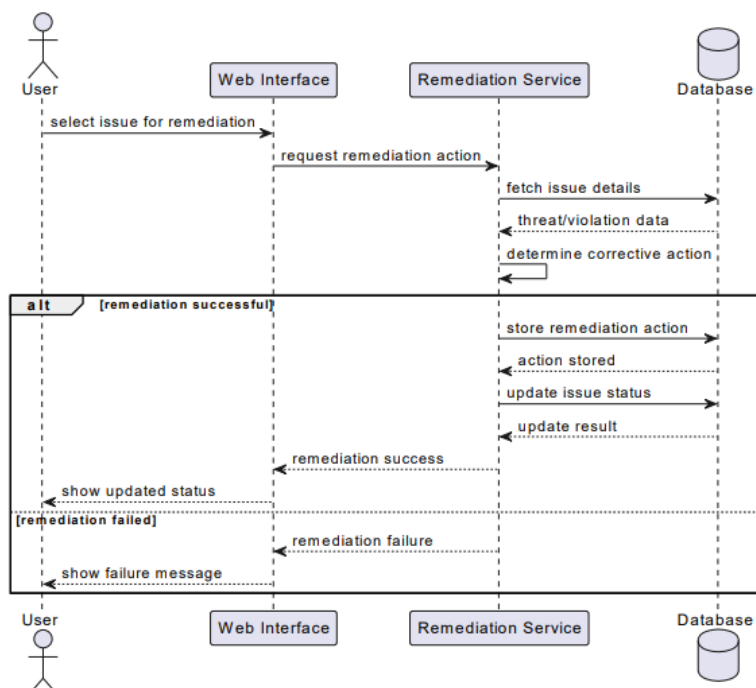


Figure 4.12: Sequence Diagram for Execute Remediation

Figure 4.13 illustrates the cost-forecasting process. The Cost Forecast Service retrieves usage information, calculates projected expenses, stores forecast records, and presents estimated cost trends.

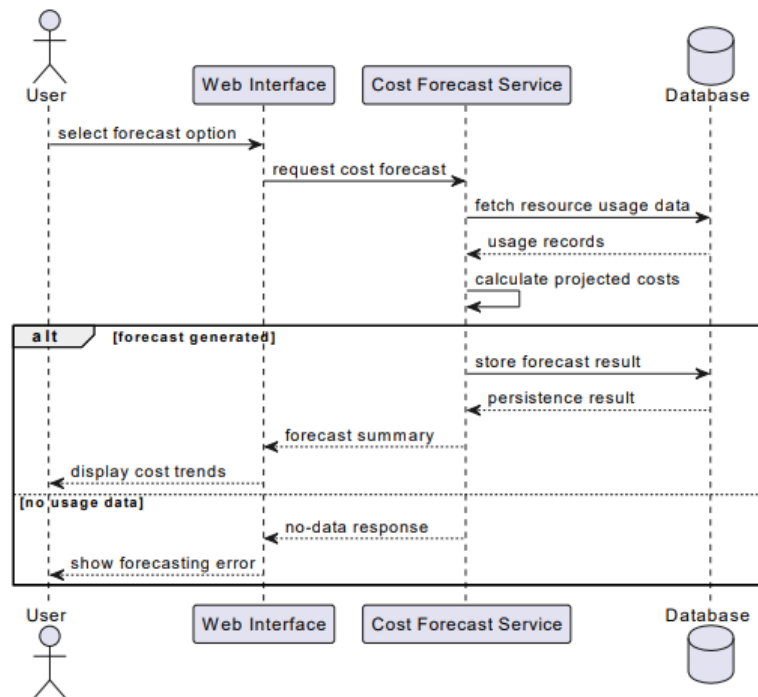


Figure 4.13: Sequence Diagram for Forecast Costs

Figure 4.14 demonstrates how reports are generated. The Report Service retrieves operational information, compiles report content, stores generated reports, and provides the final report for viewing or download.

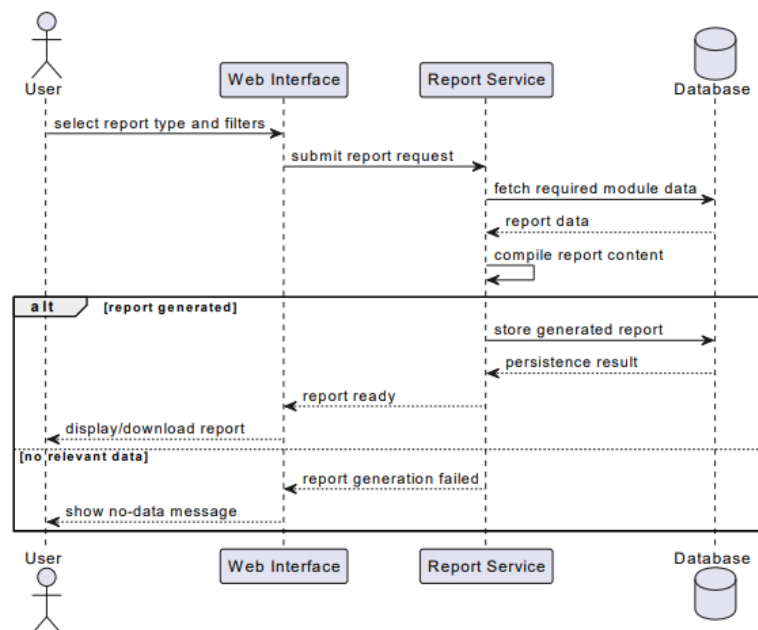


Figure 4.14: Sequence Diagram for Generate Reports

Figure 4.15 shows how user preferences are managed. The Settings Service validates submitted changes, updates stored settings, and applies the new preferences to future sessions.

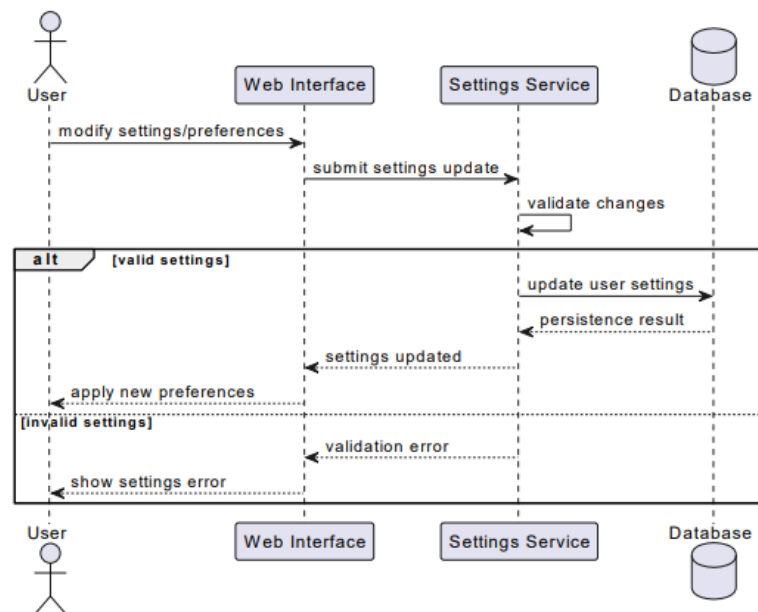


Figure 4.15: Sequence Diagram for Manage Settings

# Chapter 5

## Interfaces and Physical Design

Interfaces and physical design represent the visual and structural components of a software system that enable user interaction and data management. Interface design focuses on how users communicate with the system through screens, forms, dashboards, and controls, whereas physical design focuses on how information is organized and stored within the database. Both aspects are important because they directly influence usability, maintainability, performance, and overall system effectiveness. A well-designed interface improves user experience and operational efficiency, while a properly designed database structure ensures data consistency, integrity, and scalability. In the proposed Cloud Policy, Cost and Security Simulator, the interface layer supports educational cloud-learning activities through interactive modules, while the physical design supports the storage and management of authentication records, simulation data, governance information, monitoring metrics, forecasts, reports, and learning resources. Together, these components provide the practical foundation required for system operation and user interaction [19].

This chapter presents the interface design and physical database design of the proposed system. Section 5.1 describes the user interfaces that support authentication, organization management, simulation activities, monitoring, governance enforcement, forecasting, reporting, and personalization. Section 5.2 presents the user tables derived from the final Entity-Relationship Diagram and explains how the stored data supports the operational and educational functions of the system.

### 5.1 User Interfaces

A user interface is the visible and interactive component of a software system through which users communicate with application functionalities. It provides mechanisms for entering information, viewing system responses, navigating between modules, and performing operational tasks. User interfaces play an important role in software development because they influence usability, efficiency, learnability, and overall user satisfaction. A well-designed interface reduces complexity, improves accessibility, and enables users to complete tasks accurately with minimal effort. In educational cloud platforms, interface design becomes particularly important because users possess different levels of technical knowledge and require clear visual guidance when performing simulation and learning activities. The proposed Cloud Policy, Cost and Security Simulator utilizes a web-based user interface that supports authentication, organization management, cloud-resource



simulation, monitoring, governance, cost forecasting, reporting, and learning activities through an organized and user-friendly environment [20][21][23].

Figure 5.1 presents the Sign Up screen of the proposed system. The interface allows new users to create accounts by providing registration information and authentication details. The screen supports secure onboarding and enables users to access the cloud-learning platform.

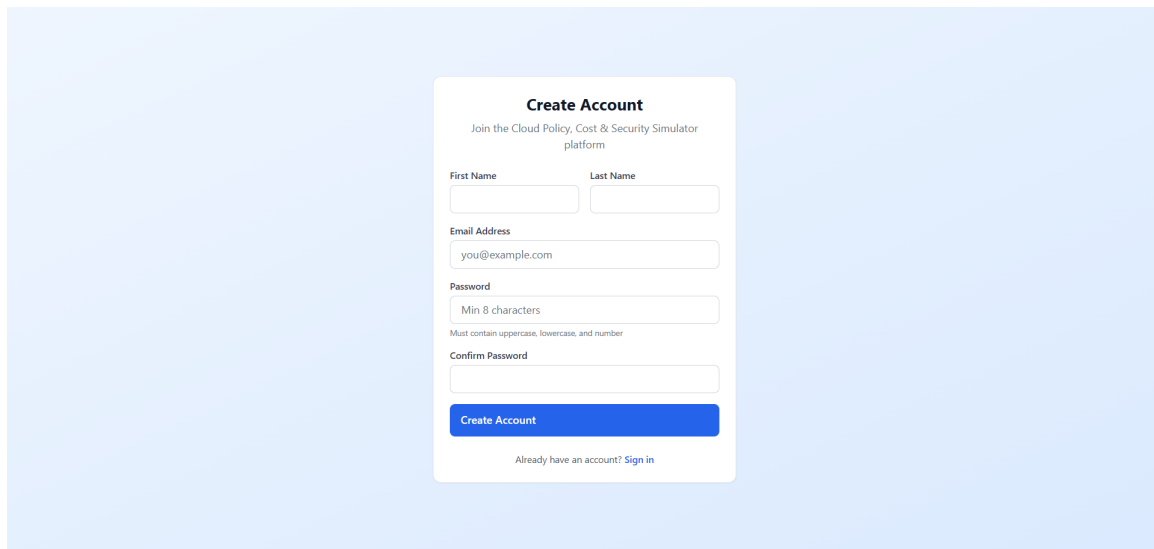
The image shows a 'Create Account' form centered on a light blue background. The form is white with a blue header 'Create Account' and a subtitle 'Join the Cloud Policy, Cost & Security Simulator platform'. It contains input fields for 'First Name', 'Last Name', 'Email Address' (with the placeholder 'you@example.com'), 'Password' (with a hint 'Min 8 characters' and a requirement 'Must contain uppercase, lowercase, and number'), and 'Confirm Password'. A blue 'Create Account' button is at the bottom, with a link 'Already have an account? Sign in' below it.

Figure 5.1: Sign Up Screen

Figure 5.2 illustrates the Login screen of the proposed system. The interface allows registered users to enter authentication credentials and gain access to authorized modules. It supports secure system access and user identification.

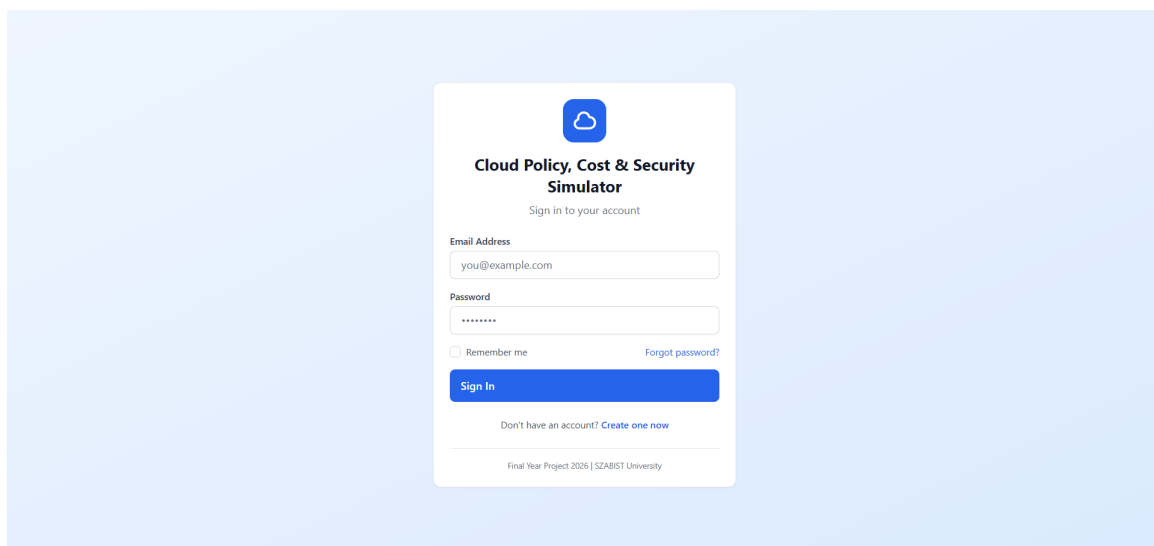
The image shows a login form for the 'Cloud Policy, Cost & Security Simulator'. It features a blue cloud icon at the top, followed by the title and subtitle. The form includes input fields for 'Email Address' (placeholder 'you@example.com') and 'Password' (masked with dots). There is a 'Remember me' checkbox and a 'Forgot password?' link. A blue 'Sign In' button is prominent. At the bottom, there is a link 'Don't have an account? Create one now' and a footer 'Final Year Project 2026 | SZABIST University'.

Figure 5.2: Account Login Screen

Figure 5.3 presents the Recover Password screen of the proposed system. The interface allows users to request password-reset functionality through a secure verification process. It improves account accessibility while maintaining authentication security.

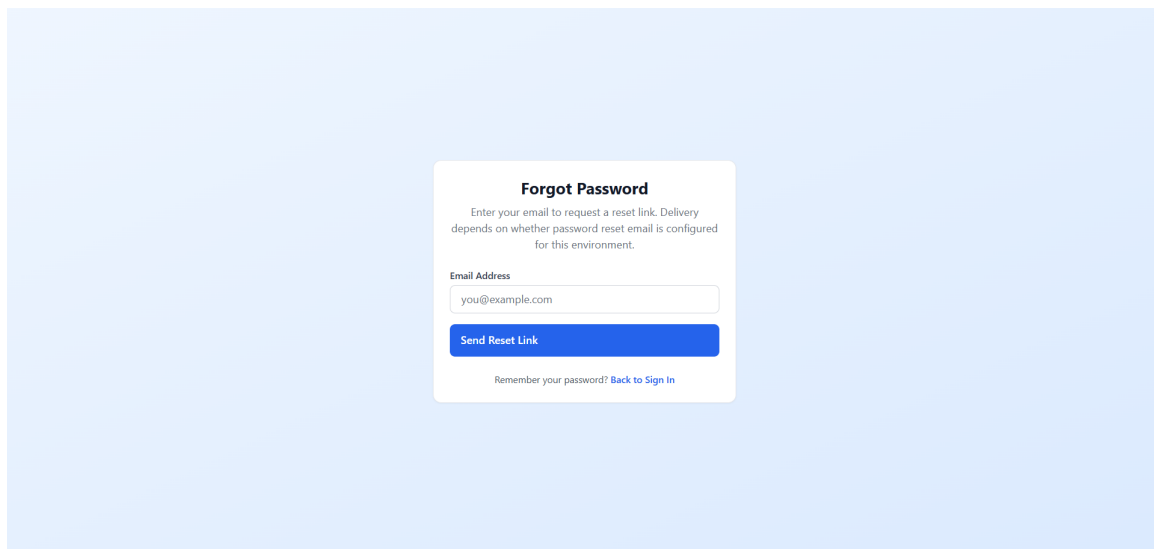


Figure 5.3: Recover Password Screen

Figure 5.4 illustrates the Dashboard screen of the proposed system. The dashboard provides users with a centralized view of simulation activities, monitoring summaries, alerts, forecasts, and recent operational information. The screen acts as the primary navigation point within the platform.

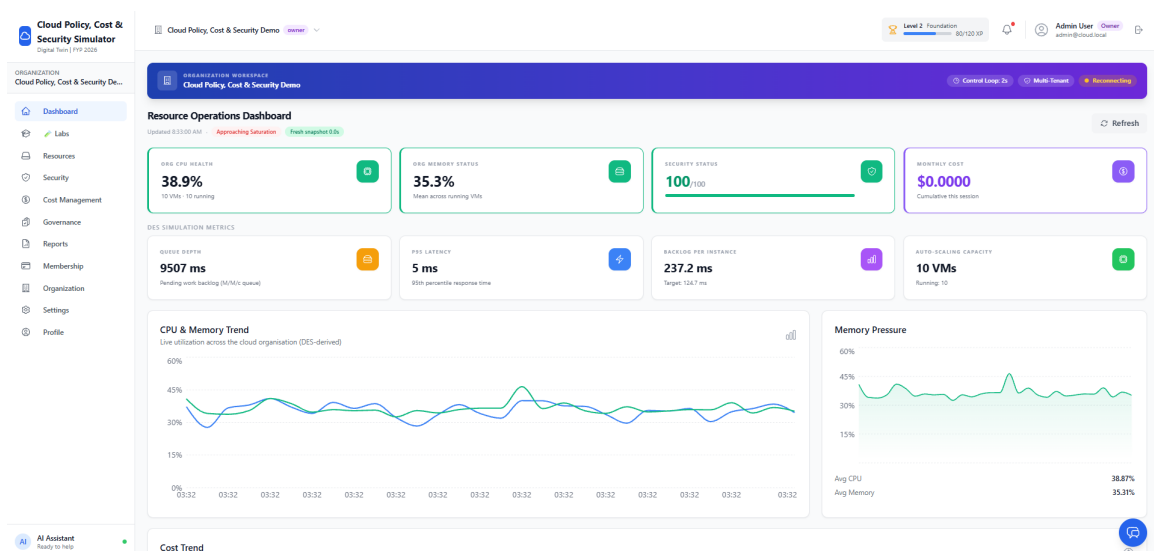


Figure 5.4: Dashboard Screen

Figure 5.5 presents the Organization Manager screen. The interface allows users to create organizations, manage organization information, and maintain tenant-specific workspaces. It supports collaborative learning and organizational resource isolation.

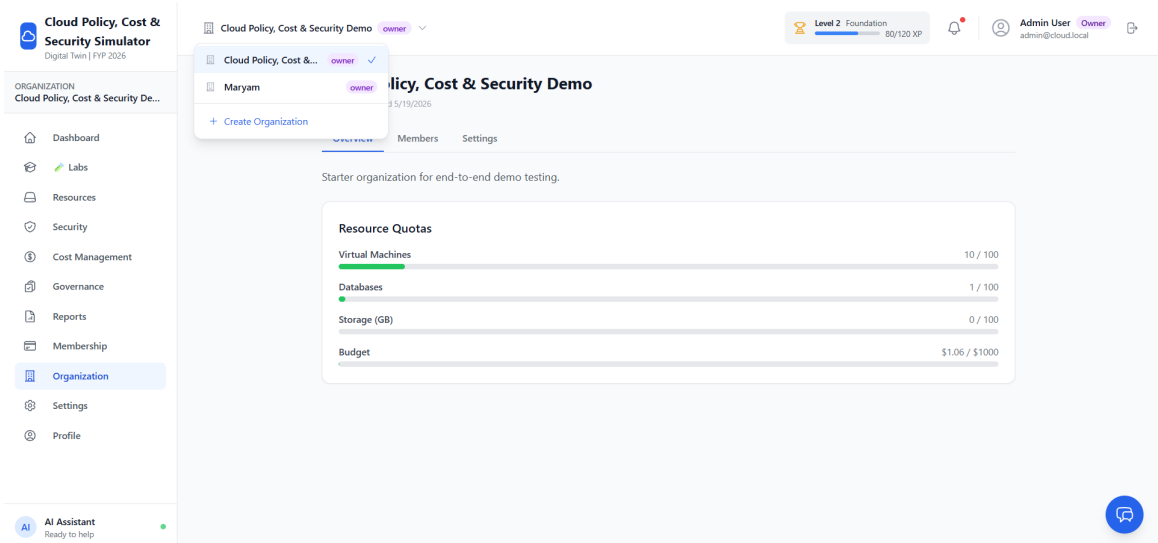


Figure 5.5: Organization Manager Screen

Figure 5.6 illustrates the Membership Management screen. The interface enables organization administrators to invite members, assign roles, and manage organizational access permissions. The screen supports secure multi-user collaboration.

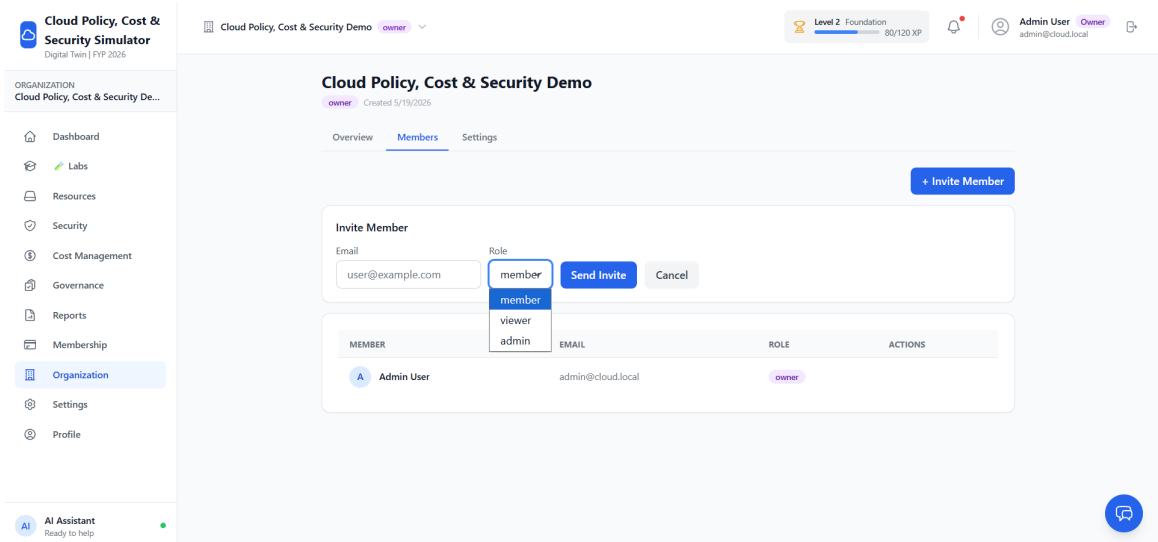


Figure 5.6: Membership Management Screen

Figure 5.7 presents the Data Generator screen. The interface allows users to generate synthetic cloud-resource metrics, telemetry records, and workload information. The generated data supports simulation, monitoring, forecasting, and threat-analysis activities.

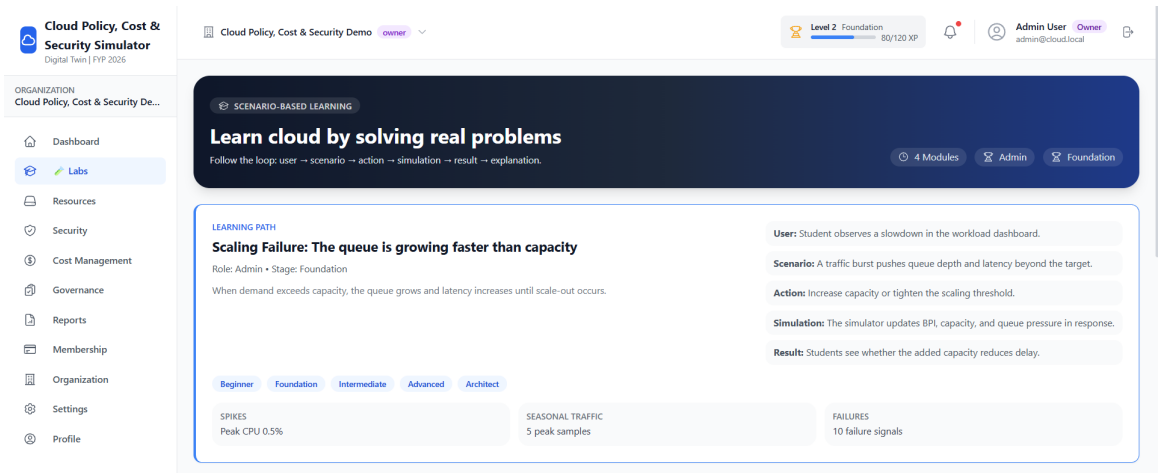


Figure 5.7: Data Generator Screen

Figure 5.8 illustrates the Resource Simulator screen. The interface enables users to create simulated virtual machines and databases within a safe educational environment. It provides practical exposure to cloud-resource management without requiring real cloud infrastructure.

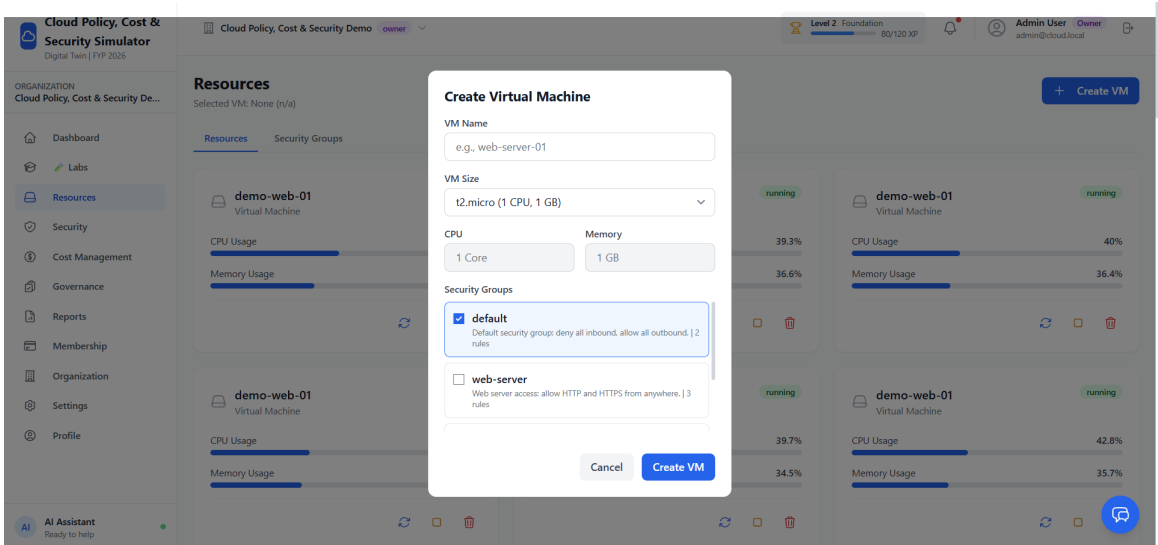


Figure 5.8: Resource Simulator Screen

Figure 5.9 presents the Resource Viewer screen. The interface displays resource information, utilization metrics, operational status, and activity summaries through visual dashboards when clicking an existing VM. It supports understanding of cloud-resource behavior and performance.

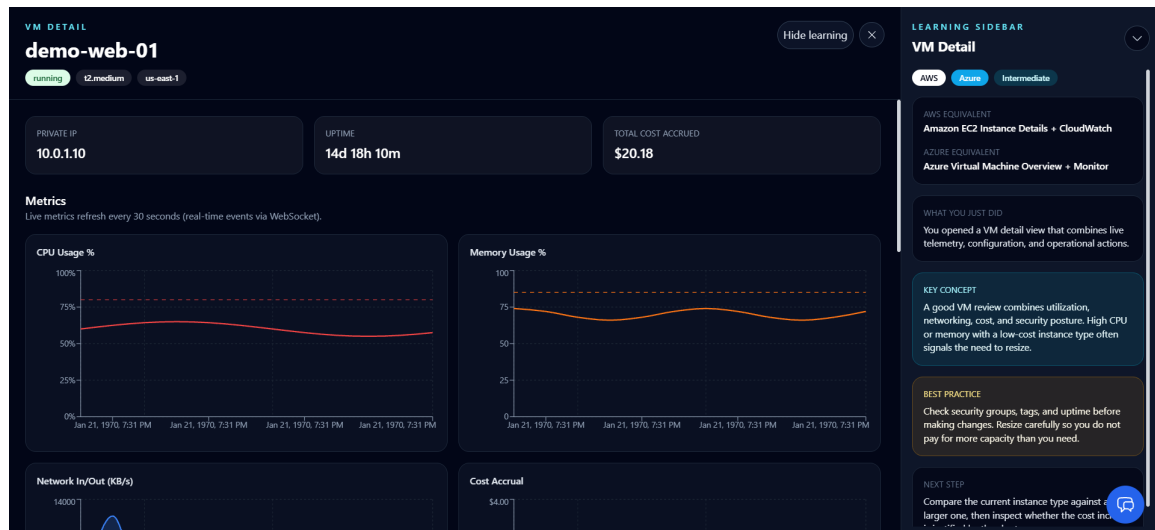


Figure 5.9: Resource Viewer Screen

Figure 5.10 illustrates the Threat Detection screen. The interface presents detected threats, suspicious activities, alerts, and security summaries generated from simulated telemetry information. It supports cybersecurity-awareness learning activities.

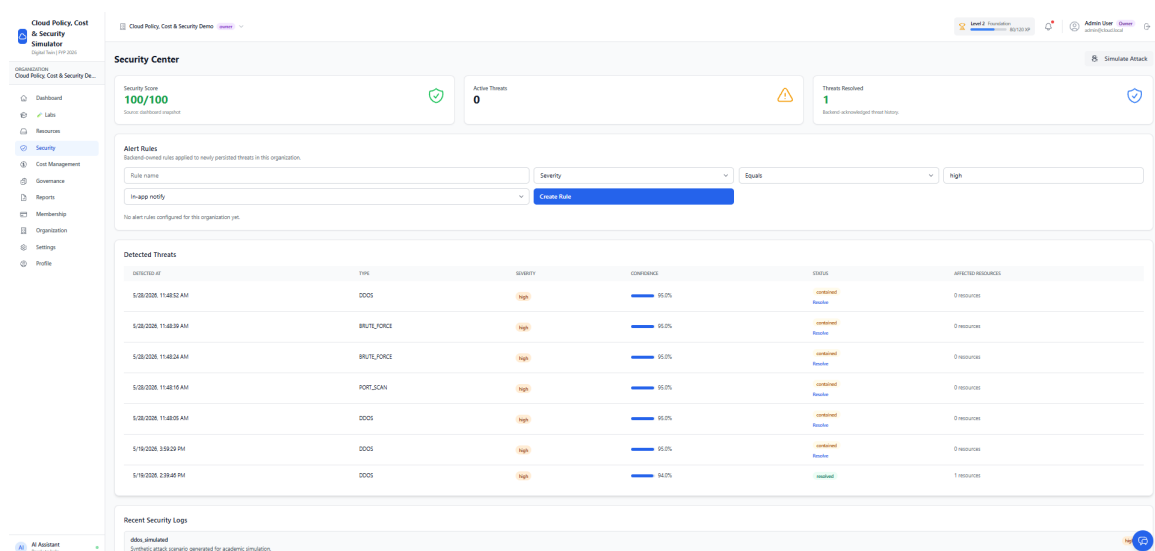


Figure 5.10: Threat Detection Screen

Figure 5.11 presents the Policy Manager screen. The interface enables users to create governance policies, manage policy definitions, and review compliance information. It supports governance education and policy-enforcement workflows.

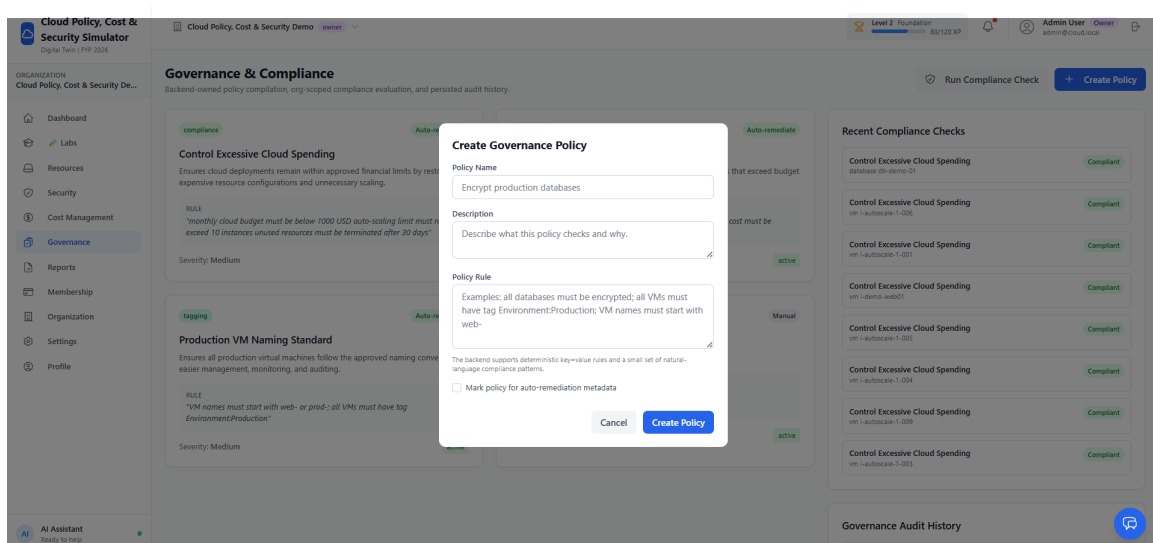


Figure 5.11: Policy Manager Screen

Figure 5.12 illustrates the Cost Forecast screen. The interface presents estimated cloud-resource costs, budget summaries, and utilization trends through visual forecasting dashboards. It supports FinOps learning and budget-management activities.

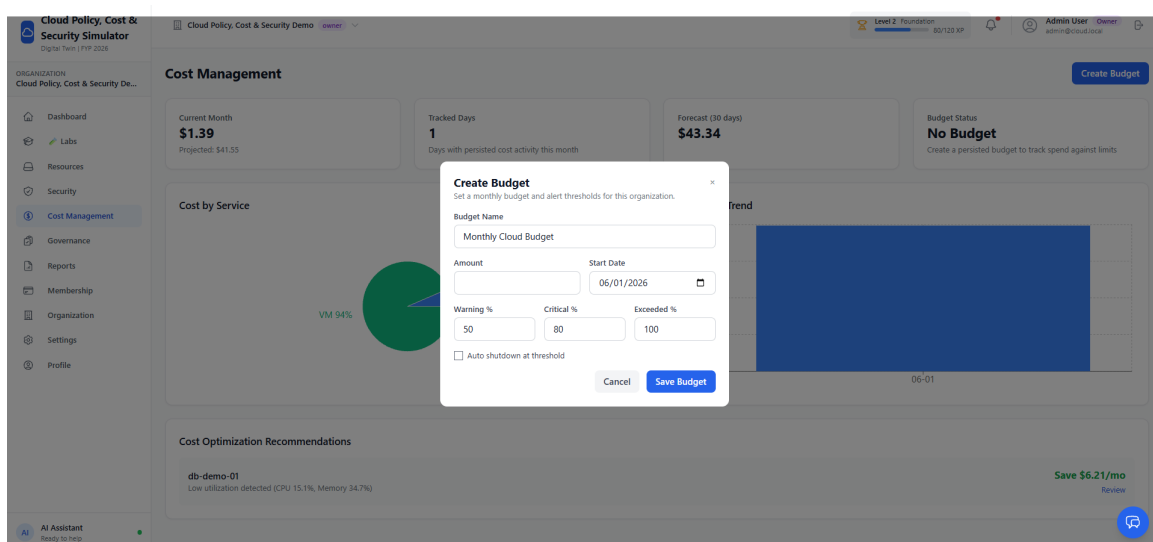


Figure 5.12: Cost Forecast Screen

Figure 5.13 presents the Report Generator screen. The interface allows users to generate governance, threat, resource, and forecasting reports. The generated reports provide structured summaries for operational analysis and learning activities.

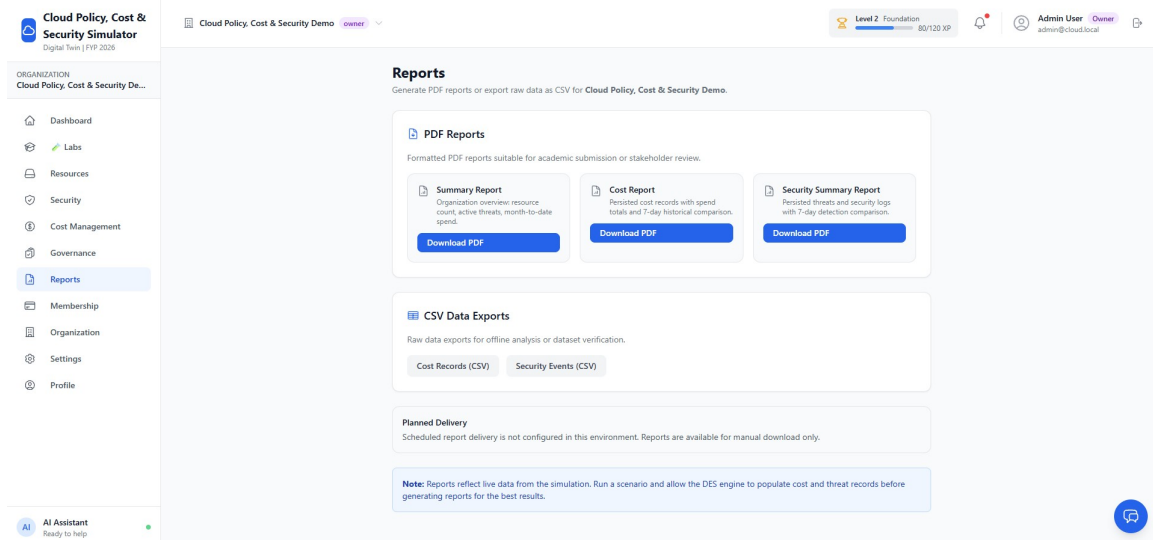


Figure 5.13: Report Generator Screen

Figure 5.14 illustrates the Learning Lab screen. The interface provides guided educational exercises, cloud-learning scenarios, and practical simulation activities. It supports self-paced learning and technical skill development.

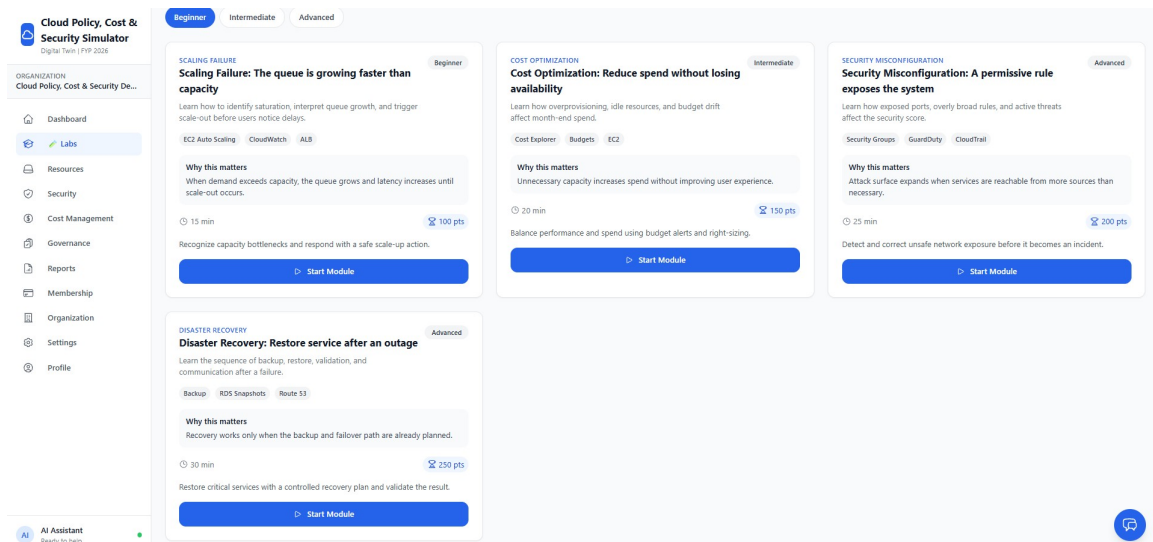


Figure 5.14: Learning Lab Screen

Figure 5.15 presents the Settings screen. The interface enables users to manage profile information, notification preferences, and personalization options. It improves usability by allowing users to customize the platform according to individual preferences.

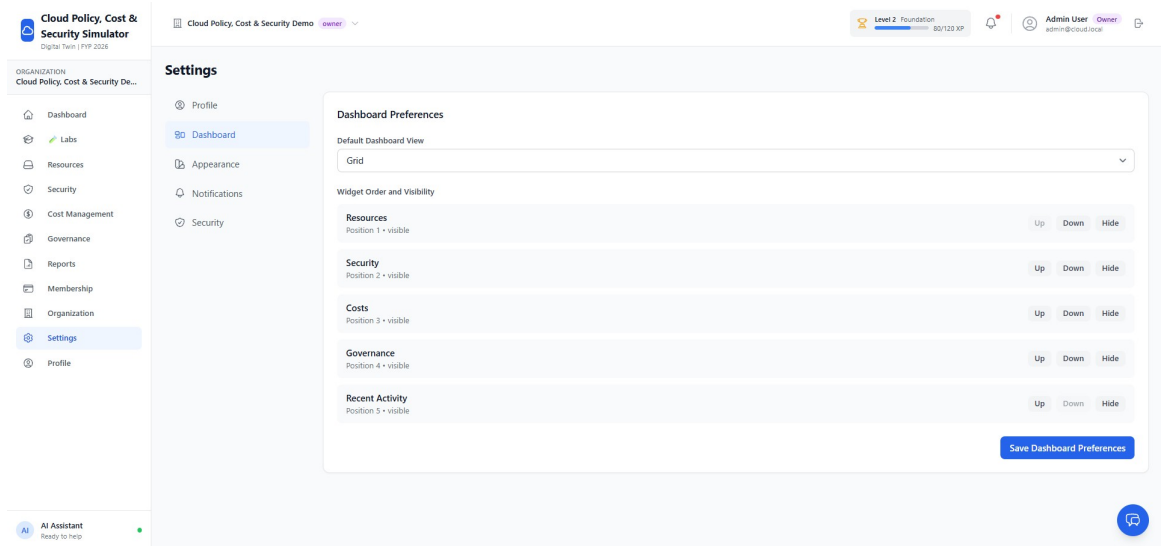


Figure 5.15: Settings Screen

## 5.2 User Tables

A user table is a structured database component that stores persistent information required for software operation. Physical database tables represent entities, relationships, transactions, and operational records that support application functionality. Proper table design improves data integrity, reduces redundancy, supports scalability, and facilitates efficient information retrieval. Each table serves a specific purpose and contributes to the overall operation of the software system. In the proposed Cloud Policy, Cost and Security Simulator, the physical database design supports authentication management, organization administration, simulation activities, governance enforcement, monitoring operations, forecasting processes, reporting services, and educational learning modules. The database structure therefore provides the implementation-level foundation necessary for supporting all major system functionalities [3].

Table 5.1 defines the User table, which stores account information for all registered users of the proposed system.

Table 5.1: User

| Field Name | Description        | Type    | Length | Default  | Not Null |
|------------|--------------------|---------|--------|----------|----------|
| user_id    | Unique user ID     | int     | 11     | identify | Y        |
| full_name  | User full name     | varchar | 90     | null     | Y        |
| email      | User email address | varchar | 120    | null     | Y        |

*Continued on next page*



Table 5.1 – Continued from previous page

| Field Name     | Description               | Type    | Length | Default | Not Null |
|----------------|---------------------------|---------|--------|---------|----------|
| password_hash  | Encrypted user password   | varchar | 255    | null    | Y        |
| email_verified | Email verification status | boolean | 1      | false   | Y        |

| Keys       |           |          |
|------------|-----------|----------|
| Table Name | Fieldname | Key Type |
| tblUser    | user_id   | Primary  |

Table 5.2 defines the Settings table of the proposed system. The table stores user preferences such as theme, default view, and alert preferences.

Table 5.2: Settings

| Field Name       | Description            | Type   | Length | Default   | Not Null |
|------------------|------------------------|--------|--------|-----------|----------|
| settings_id      | Unique settings ID     | Int64  | 11     | identify  | Y        |
| user_id          | Reference to User      | Int64  | 11     | identify  | Y        |
| theme            | Interface theme        | String | 50     | default   | Y        |
| default_view     | Default dashboard view | String | 90     | dashboard | Y        |
| alert_preference | User alert preference  | String | 90     | normal    | Y        |

| Keys        |             |          |
|-------------|-------------|----------|
| Table Name  | Fieldname   | Key Type |
| tblSettings | settings_id | Primary  |
| tblUser     | user_id     | Foreign  |

Table 5.3 defines the PasswordResetToken table of the proposed system. The table stores reset tokens used to support secure password-recovery operations.

Table 5.3: PasswordReset

| Field Name | Description            | Type      | Length | Default  | Not Null |
|------------|------------------------|-----------|--------|----------|----------|
| reset_id   | Unique reset ID        | Int64     | 11     | identify | Y        |
| user_id    | Reference to User      | Int64     | 11     | identify | Y        |
| token      | Password reset token   | String    | 255    | identify | Y        |
| expires_at | Token expiry date time | Timestamp | 20     | identify | Y        |
| used       | Token usage status     | Boolean   | 1      | false    | Y        |

| Keys                  |           |          |
|-----------------------|-----------|----------|
| Table Name            | Fieldname | Key Type |
| tblPasswordResetToken | reset_id  | Primary  |
| tblUser               | user_id   | Foreign  |

Table 5.4 defines the EmailVerificationToken table of the proposed system. The table stores verification tokens used during account activation and email confirmation.

Table 5.4: EmailVerification

| Field Name      | Description                  | Type      | Length | Default  | Not Null |
|-----------------|------------------------------|-----------|--------|----------|----------|
| verification_id | Unique verification ID       | Int64     | 11     | identify | Y        |
| user_id         | Reference to User            | Int64     | 11     | identify | Y        |
| token           | Email verification token     | String    | 255    | identify | Y        |
| expires_at      | Token expiry date time       | Timestamp | 20     | identify | Y        |
| verified_at     | Verification completion time | Timestamp | 20     | identify | N        |

| Keys                      |                 |          |
|---------------------------|-----------------|----------|
| Table Name                | Fieldname       | Key Type |
| tblEmailVerificationToken | verification_id | Primary  |
| tblUser                   | user_id         | Foreign  |

Table 5.5 defines the Notification table of the proposed system. The table stores messages delivered to users regarding alerts, reports, activities, and system updates.

Table 5.5: Notification

| Field Name      | Description                | Type      | Length | Default  | Not Null |
|-----------------|----------------------------|-----------|--------|----------|----------|
| notification_id | Unique notification ID     | Int64     | 11     | identify | Y        |
| user_id         | Reference to User          | Int64     | 11     | identify | Y        |
| type            | Notification type          | String    | 60     | system   | Y        |
| is_read         | Read status                | Boolean   | 1      | false    | Y        |
| created_at      | Notification creation time | Timestamp | 20     | current  | Y        |

| Keys            |                 |          |
|-----------------|-----------------|----------|
| Table Name      | Fieldname       | Key Type |
| tblNotification | notification_id | Primary  |
| tblUser         | user_id         | Foreign  |

Table 5.6 defines the AuditTrail table of the proposed system. The table stores user activities and system events for accountability and monitoring purposes.

Table 5.6: AuditTrail

| Field Name  | Description       | Type      | Length | Default  | Not Null |
|-------------|-------------------|-----------|--------|----------|----------|
| audit_id    | Unique audit ID   | Int64     | 11     | identify | Y        |
| user_id     | Reference to User | Int64     | 11     | identify | Y        |
| action      | Performed action  | String    | 90     | identify | Y        |
| action_time | Action date time  | Timestamp | 20     | current  | Y        |

| Keys          |           |          |
|---------------|-----------|----------|
| Table Name    | Fieldname | Key Type |
| tblAuditTrail | audit_id  | Primary  |
| tblUser       | user_id   | Foreign  |

Table 5.7 defines the UserRole table of the proposed system. The table stores role information used to manage access control and permission levels.

Table 5.7: UserRole

| Field Name            | Description    | Type    | Length | Default  | Not Null |
|-----------------------|----------------|---------|--------|----------|----------|
| role <sub>i</sub> d   | Unique role ID | int     | 11     | identify | Y        |
| role <sub>n</sub> ame | Role name      | varchar | 90     | null     | Y        |

| Keys        |           |          |
|-------------|-----------|----------|
| Table Name  | Fieldname | Key Type |
| tblUserRole | role_id   | Primary  |

Table 5.8 defines the Organization table of the proposed system. The table stores organization workspace information and supports tenant-based resource management.

Table 5.8: Organization

| Field Name | Description            | Type    | Length | Default  | Not Null |
|------------|------------------------|---------|--------|----------|----------|
| org_id     | Unique organization ID | int     | 11     | identify | Y        |
| org_name   | Organization name      | varchar | 90     | null     | Y        |
| owner_id   | Organization owner ID  | int     | 11     | null     | Y        |

| Keys            |           |          |
|-----------------|-----------|----------|
| Table Name      | Fieldname | Key Type |
| tblOrganization | org_id    | Primary  |
| tblUser         | owner_id  | Foreign  |

Table 5.9 defines the Invitation table of the proposed system. The table stores invitation records used to add users into organization workspaces.

Table 5.9: Invitation

| Field Name    | Description               | Type      | Length | Default  | Not Null |
|---------------|---------------------------|-----------|--------|----------|----------|
| invitation_id | Unique invitation ID      | int       | 11     | identify | Y        |
| org_id        | Reference to Organization | int       | 11     | null     | Y        |
| invited_email | Invited user email        | varchar   | 120    | null     | Y        |
| role_id       | Reference to UserRole     | int       | 11     | null     | Y        |
| invited_by    | Inviter user ID           | int       | 11     | null     | Y        |
| status        | Invitation status         | varchar   | 50     | pending  | Y        |
| sent_at       | Invitation sent time      | timestamp | 20     | current  | Y        |

| Keys            |               |          |
|-----------------|---------------|----------|
| Table Name      | Fieldname     | Key Type |
| tblInvitation   | invitation_id | Primary  |
| tblOrganization | org_id        | Foreign  |
| tblUserRole     | role_id       | Foreign  |
| tblUser         | invited_by    | Foreign  |

Table 5.10 defines the Membership table of the proposed system. The table connects users with organizations and supports role-based workspace access.

Table 5.10: Membership

| Field Name    | Description               | Type    | Length | Default  | Not Null |
|---------------|---------------------------|---------|--------|----------|----------|
| membership_id | Unique membership ID      | int     | 11     | identify | Y        |
| user_id       | Reference to User         | int     | 11     | null     | Y        |
| org_id        | Reference to Organization | int     | 11     | null     | Y        |
| role_id       | Reference to UserRole     | int     | 11     | null     | Y        |
| status        | Membership status         | varchar | 50     | active   | Y        |

| Keys            |               |          |
|-----------------|---------------|----------|
| Table Name      | Fieldname     | Key Type |
| tblMembership   | membership_id | Primary  |
| tblUser         | user_id       | Foreign  |
| tblOrganization | org_id        | Foreign  |
| tblUserRole     | role_id       | Foreign  |

Table 5.11 defines the BaselineProfile table of the proposed system. The table stores workload profiles used for simulation and synthetic-data generation.

Table 5.11: BaselineProfile

| Field Name    | Description               | Type      | Length | Default  | Not Null |
|---------------|---------------------------|-----------|--------|----------|----------|
| baseline_id   | Unique baseline ID        | int       | 11     | identify | Y        |
| org_id        | Reference to Organization | int       | 11     | null     | Y        |
| baseline_name | Baseline profile name     | varchar   | 90     | null     | Y        |
| created_at    | Profile creation time     | timestamp | 20     | current  | Y        |

| Keys               |             |          |
|--------------------|-------------|----------|
| Table Name         | Fieldname   | Key Type |
| tblBaselineProfile | baseline_id | Primary  |
| tblOrganization    | org_id      | Foreign  |

Table 5.12 defines the SimulationRun table of the proposed system. The table stores simulation execution records and tracks simulation scenario status.

Table 5.12: SimulationRun

| Field Name    | Description                  | Type      | Length | Default  | Not Null |
|---------------|------------------------------|-----------|--------|----------|----------|
| simulation_id | Unique simulation ID         | int       | 11     | identify | Y        |
| org_id        | Reference to Organization    | int       | 11     | null     | Y        |
| baseline_id   | Reference to BaselineProfile | int       | 11     | null     | Y        |
| scenario_name | Simulation scenario name     | varchar   | 90     | null     | Y        |
| started_at    | Simulation start time        | timestamp | 20     | null     | Y        |
| ended_at      | Simulation end time          | timestamp | 20     | null     | N        |
| status        | Simulation status            | varchar   | 50     | running  | Y        |

| Keys               |               |          |
|--------------------|---------------|----------|
| Table Name         | Fieldname     | Key Type |
| tblSimulationRun   | simulation_id | Primary  |
| tblOrganization    | org_id        | Foreign  |
| tblBaselineProfile | baseline_id   | Foreign  |

Table 5.13 defines the CostForecast table of the proposed system. The table stores predicted cost records generated from simulated cloud-resource activity.

Table 5.13: CostForecast

| Field Name     | Description               | Type  | Length | Default  | Not Null |
|----------------|---------------------------|-------|--------|----------|----------|
| forecast_id    | Unique forecast ID        | int   | 11     | identify | Y        |
| org_id         | Reference to Organization | int   | 11     | null     | Y        |
| predicted_cost | Forecasted cost value     | float | 12     | 0        | Y        |
| forecast_date  | Forecast date             | date  | 20     | current  | Y        |

| Keys            |             |          |
|-----------------|-------------|----------|
| Table Name      | Fieldname   | Key Type |
| tblCostForecast | forecast_id | Primary  |
| tblOrganization | org_id      | Foreign  |

Table 5.14 defines the Budget table of the proposed system. The table stores budget limits and budget periods used for FinOps and cost-management activities.

Table 5.14: Budget

| Field Name   | Description               | Type    | Length | Default  | Not Null |
|--------------|---------------------------|---------|--------|----------|----------|
| budget_id    | Unique budget ID          | int     | 11     | identify | Y        |
| org_id       | Reference to Organization | int     | 11     | null     | Y        |
| budget_name  | Budget title              | varchar | 90     | null     | Y        |
| amount_limit | Budget limit amount       | float   | 12     | 0        | Y        |
| period       | Budget period             | varchar | 50     | monthly  | Y        |

| Keys            |           |          |
|-----------------|-----------|----------|
| Table Name      | Fieldname | Key Type |
| tblBudget       | budget_id | Primary  |
| tblOrganization | org_id    | Foreign  |

Table 5.15 defines the Policy table of the proposed system. The table stores governance-policy rules used to evaluate simulated cloud resources.

Table 5.15: Policy

| Field Name  | Description               | Type    | Length | Default  | Not Null |
|-------------|---------------------------|---------|--------|----------|----------|
| policy_id   | Unique policy ID          | int     | 11     | identify | Y        |
| org_id      | Reference to Organization | int     | 11     | null     | Y        |
| policy_name | Policy name               | varchar | 90     | null     | Y        |
| policy_type | Policy category           | varchar | 90     | null     | Y        |

| Keys            |           |          |
|-----------------|-----------|----------|
| Table Name      | Fieldname | Key Type |
| tblPolicy       | policy_id | Primary  |
| tblOrganization | org_id    | Foreign  |

Table 5.16 defines the SimulatedResource table of the proposed system. The table stores virtual machines, databases, and other simulated cloud resources.

Table 5.16: SimulatedResource

| Field Name    | Description               | Type      | Length | Default  | Not Null |
|---------------|---------------------------|-----------|--------|----------|----------|
| resource_id   | Unique resource ID        | int       | 11     | identify | Y        |
| org_id        | Reference to Organization | int       | 11     | null     | Y        |
| resource_name | Resource name             | varchar   | 90     | null     | Y        |
| resource_type | Resource type             | varchar   | 90     | null     | Y        |
| status        | Resource status           | varchar   | 50     | active   | Y        |
| created_at    | Resource creation time    | timestamp | 20     | current  | Y        |

| Keys                 |             |          |
|----------------------|-------------|----------|
| Table Name           | Fieldname   | Key Type |
| tblSimulatedResource | resource_id | Primary  |
| tblOrganization      | org_id      | Foreign  |

Table 5.17 defines the PolicyViolation table of the proposed system. The table stores policy violations identified during resource-governance evaluation.

Table 5.17: PolicyViolation

| Field Name     | Description                    | Type    | Length | Default  | Not Null |
|----------------|--------------------------------|---------|--------|----------|----------|
| violation_id   | Unique violation ID            | int     | 11     | identify | Y        |
| policy_id      | Reference to Policy            | int     | 11     | null     | Y        |
| resource_id    | Reference to SimulatedResource | int     | 11     | null     | Y        |
| violation_date | Violation date                 | date    | 20     | current  | Y        |
| severity       | Violation severity             | varchar | 50     | medium   | Y        |

| Keys               |              |          |
|--------------------|--------------|----------|
| Table Name         | Fieldname    | Key Type |
| tblPolicyViolation | violation_id | Primary  |
| tblPolicy          | policy_id    | Foreign  |

*Continued on next page*

*Continued from previous page*

| Keys                 |             |          |
|----------------------|-------------|----------|
| Table Name           | Fieldname   | Key Type |
| tblSimulatedResource | resource_id | Foreign  |

Table 5.18 defines the TrafficLog table of the proposed system. The table stores simulated traffic records used for monitoring and threat-detection workflows.

Table 5.18: TrafficLog

| Field Name   | Description                    | Type      | Length | Default  | Not Null |
|--------------|--------------------------------|-----------|--------|----------|----------|
| traffic_id   | Unique traffic ID              | int       | 11     | identify | Y        |
| resource_id  | Reference to SimulatedResource | int       | 11     | null     | Y        |
| traffic_type | Traffic category               | varchar   | 90     | null     | Y        |
| volume       | Traffic volume                 | float     | 12     | 0        | Y        |
| recorded_at  | Traffic recorded time          | timestamp | 20     | current  | Y        |

| Keys                 |             |          |
|----------------------|-------------|----------|
| Table Name           | Fieldname   | Key Type |
| tblTrafficLog        | traffic_id  | Primary  |
| tblSimulatedResource | resource_id | Foreign  |

Table 5.19 defines the ResourceMetric table of the proposed system. The table stores CPU and memory usage values generated by simulated resources.

Table 5.19: ResourceMetric

| Field Name   | Description                    | Type      | Length | Default  | Not Null |
|--------------|--------------------------------|-----------|--------|----------|----------|
| metric_id    | Unique metric ID               | int       | 11     | identify | Y        |
| resource_id  | Reference to SimulatedResource | int       | 11     | null     | Y        |
| cpu_usage    | CPU utilization value          | float     | 12     | 0        | Y        |
| memory_usage | Memory utilization value       | float     | 12     | 0        | Y        |
| recorded_at  | Metric recorded time           | timestamp | 20     | current  | Y        |

| Keys                 |             |          |
|----------------------|-------------|----------|
| Table Name           | Fieldname   | Key Type |
| tblResourceMetric    | metric_id   | Primary  |
| tblSimulatedResource | resource_id | Foreign  |

Table 5.20 defines the ThreatLog table of the proposed system. The table stores detected threats generated from traffic and resource-analysis activities.



Table 5.20: ThreatLog

| Field Name  | Description                    | Type      | Length | Default  | Not Null |
|-------------|--------------------------------|-----------|--------|----------|----------|
| threat_id   | Unique threat ID               | int       | 11     | identify | Y        |
| org_id      | Reference to Organization      | int       | 11     | null     | Y        |
| resource_id | Reference to SimulatedResource | int       | 11     | null     | Y        |
| threat_type | Threat category                | varchar   | 90     | null     | Y        |
| severity    | Threat severity level          | varchar   | 50     | medium   | Y        |
| detected_at | Detection date time            | timestamp | 20     | current  | Y        |

| Keys                 |             |          |
|----------------------|-------------|----------|
| Table Name           | Fieldname   | Key Type |
| tblThreatLog         | threat_id   | Primary  |
| tblOrganization      | org_id      | Foreign  |
| tblSimulatedResource | resource_id | Foreign  |

Table 5.21 defines the Alert table of the proposed system. The table stores alert messages generated from detected threats and operational issues.

Table 5.21: Alert

| Field Name | Description            | Type      | Length | Default  | Not Null |
|------------|------------------------|-----------|--------|----------|----------|
| alert_id   | Unique alert ID        | int       | 11     | identify | Y        |
| threat_id  | Reference to ThreatLog | int       | 11     | null     | Y        |
| user_id    | Reference to User      | int       | 11     | null     | Y        |
| alert_type | Alert category         | varchar   | 90     | null     | Y        |
| status     | Alert status           | varchar   | 50     | unread   | Y        |
| created_at | Alert creation time    | timestamp | 20     | current  | Y        |

| Keys         |           |          |
|--------------|-----------|----------|
| Table Name   | Fieldname | Key Type |
| tblAlert     | alert_id  | Primary  |
| tblThreatLog | threat_id | Foreign  |
| tblUser      | user_id   | Foreign  |

Table 5.22 defines the RemediationAction table of the proposed system. The table stores corrective actions applied to threats or governance violations.

Table 5.22: RemediationAction

| Field Name     | Description                    | Type      | Length | Default  | Not Null |
|----------------|--------------------------------|-----------|--------|----------|----------|
| remediation_id | Unique remediation ID          | int       | 11     | identify | Y        |
| threat_id      | Reference to ThreatLog         | int       | 11     | null     | N        |
| resource_id    | Reference to SimulatedResource | int       | 11     | null     | N        |
| assigned_to    | Assigned user ID               | int       | 11     | null     | Y        |
| action_type    | Remediation action type        | varchar   | 90     | null     | Y        |
| status         | Remediation status             | varchar   | 50     | pending  | Y        |
| executed_at    | Execution date time            | timestamp | 20     | null     | N        |

| Keys                 |                |          |
|----------------------|----------------|----------|
| Table Name           | Fieldname      | Key Type |
| tblRemediationAction | remediation_id | Primary  |
| tblThreatLog         | threat_id      | Foreign  |
| tblSimulatedResource | resource_id    | Foreign  |
| tblUser              | assigned_to    | Foreign  |

Table 5.23 defines the LearningLab table of the proposed system. The table stores guided educational scenarios and cloud-learning activities.

Table 5.23: LearningLab

| Field Name | Description               | Type    | Length | Default  | Not Null |
|------------|---------------------------|---------|--------|----------|----------|
| lab_id     | Unique lab ID             | int     | 11     | identify | Y        |
| org_id     | Reference to Organization | int     | 11     | null     | Y        |
| title      | Lab title                 | varchar | 90     | null     | Y        |
| difficulty | Difficulty level          | varchar | 50     | beginner | Y        |

| Keys            |           |          |
|-----------------|-----------|----------|
| Table Name      | Fieldname | Key Type |
| tblLearningLab  | lab_id    | Primary  |
| tblOrganization | org_id    | Foreign  |

Table 5.24 defines the Report table of the proposed system. The table stores generated reports and export details for analysis and documentation

Table 5.24: Report

| Field Name    | Description               | Type      | Length | Default  | Not Null |
|---------------|---------------------------|-----------|--------|----------|----------|
| report_id     | Unique report ID          | int       | 11     | identify | Y        |
| org_id        | Reference to Organization | int       | 11     | null     | Y        |
| generated_by  | Report generator user ID  | int       | 11     | null     | Y        |
| report_type   | Report category           | varchar   | 90     | null     | Y        |
| export_format | Report export format      | varchar   | 50     | pdf      | Y        |
| created_at    | Report creation time      | timestamp | 20     | current  | Y        |

| Keys            |              |          |
|-----------------|--------------|----------|
| Table Name      | Fieldname    | Key Type |
| tblReport       | report_id    | Primary  |
| tblOrganization | org_id       | Foreign  |
| tblUser         | generated_by | Foreign  |

# References

- [1] P. Mell and T. Grance, “The NIST definition of cloud computing,” National Institute of Standards and Technology, Special Publication 800-145, 2011.
- [2] M. Grieves and J. Vickers, “Digital Twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017.
- [3] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [4] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA, USA: SAGE Publications, 2018.
- [5] T. Erl, R. Puttini, and Z. Mahmood, *Cloud Computing: Concepts, Technology and Architecture*. Upper Saddle River, NJ, USA: Prentice Hall, 2013.
- [6] Amazon Web Services, “Amazon CloudWatch Documentation,” AWS Documentation. [Online]. Available: <https://docs.aws.amazon.com/cloudwatch/>. [Accessed: May 2026].
- [7] Microsoft, “Monitor Azure Virtual Machines,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/azure/azure-monitor/>. [Accessed: May 2026].
- [8] Google Cloud, “Compute Engine Documentation,” Google Cloud Documentation. [Online]. Available: <https://cloud.google.com/compute/docs>. [Accessed: May 2026].
- [9] VMware, “VMware vSphere,” VMware Documentation. [Online]. Available: <https://docs.vmware.com/en/VMware-vSphere/>. [Accessed: May 2026].
- [10] Cisco Networking Academy, “Cisco Packet Tracer,” Cisco NetAcad. [Online]. Available: <https://www.netacad.com/courses/packet-tracer>. [Accessed: May 2026].
- [11] Amazon Web Services, “AWS Educate: Cloud Skills for Education,” AWS Educate. [Online]. Available: <https://aws.amazon.com/education/awseducate/>. [Accessed: May 2026].
- [12] Microsoft, “Azure Lab Services Documentation,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/azure/lab-services/>. [Accessed: May 2026].

- [13] Datadog, “Infrastructure Monitoring Documentation,” Datadog Docs. [Online]. Available: <https://docs.datadoghq.com/>. [Accessed: May 2026].
- [14] Splunk, “Splunk Observability Cloud,” Splunk Documentation. [Online]. Available: <https://docs.splunk.com/Observability/>. [Accessed: May 2026].
- [15] OpenInfra Foundation, “OpenStack: Open Source Cloud Computing Infrastructure,” OpenStack Documentation. [Online]. Available: <https://docs.openstack.org/>. [Accessed: May 2026].
- [16] K. Beck et al., “Manifesto for Agile Software Development,” Agile Alliance, 2001. [Online]. Available: <https://agilemanifesto.org/>. [Accessed: May 2026].
- [17] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.
- [18] IEEE Computer Society, “IEEE Standard for Software and System Test Documentation,” IEEE Std 829, 2008.
- [19] ISO/IEC, “Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE),” ISO/IEC 25010, 2011.
- [20] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [21] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.
- [22] Object Management Group, “Unified Modeling Language (UML) Version 2.5.1,” OMG Standard, 2017.
- [23] ISO/IEC, “Ergonomics of Human-System Interaction — Part 11: Usability: Definitions and Concepts,” ISO 9241-11, 2018.
- [24] T. Erl, R. Puttini, and Z. Mahmood, *Cloud Computing: Concepts, Technology and Architecture*. Prentice Hall, 2013.
- [25] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Addison-Wesley, 2003.
- [26] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015.
- [27] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson, 2015.
- [28] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.